Experiment No.—1: Introduction to MySQL Environment

Aim:

a) Installation of Workbench or XAMPP server
b) Create Database "College_DB"
   Check existing database
   Drop database
c) Create a new MySQL user named **college_user** with password **Password@123**.
   - Create User
   - Check if user exist
   - Grant privileges to user
   - Drop existing User

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the MySQL environment and its basic components.
2. Install and configure MySQL Workbench or XAMPP server successfully.
3. Create, view, and delete databases using SQL commands.
4. Manage MySQL users by creating and removing user accounts.
5. Assign and control user privileges to ensure secure database access.

Theory:

1. Introduction to MySQL Environment

The MySQL environment refers to the setup required to create, manage, and control databases using MySQL Server and its supporting tools such as MySQL Workbench or XAMPP.

This experiment focuses specifically on:

- Installing and configuring MySQL tools.
- Creating and managing a database named `College_DB`.
- Managing MySQL users and assigning privileges.

The practical mainly covers **database operations** and **user management operations**.

2. Installation of MySQL Workbench / XAMPP

To work with MySQL, the server must be installed and running.

2.1 MySQL Workbench

MySQL Workbench is a graphical tool used to:

- Connect to MySQL Server.
- Execute SQL queries.
- Create and delete databases.
- Manage users and privileges.

After installation:

1. Open MySQL Workbench.
2. Connect to the local MySQL Server.
3. Open a new SQL query tab to execute commands.

2.2 XAMPP Server

XAMPP includes MySQL (MariaDB) as part of its package.

Steps:

1. Install XAMPP.
2. Start Apache and MySQL from XAMPP Control Panel.
3. Open phpMyAdmin through the browser.
4. Execute SQL queries in the SQL tab.

Both tools allow performing the same SQL operations.

3. Database Management

3.1 Creating a Database

A database is a collection of related data.

To create a database:

```
CREATE DATABASE College_DB;
```

This command creates a new database named `College_DB`.

3.2 Checking Existing Databases

Before creating or deleting a database, it is important to verify existing databases.

```
SHOW DATABASES;
```

This displays all available databases in the MySQL Server.

Example Output (Conceptual Representation):

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| College_DB         |
+--------------------+
```

If `College_DB` appears in the list, it means it exists.

3.3 Dropping a Database

To delete an existing database:

```
DROP DATABASE College_DB;
```

Important Points:

- Deletes the entire database permanently.
- Removes all tables and data inside it.
- Should be executed carefully.

4. MySQL User Management

User management ensures secure access to databases. Different users can be given different permissions.

4.1 Creating a New User

Syntax:

```
CREATE USER 'college_user'@'localhost' IDENTIFIED BY
'Password@123';
```

Explanation:

`college_user` → Name of the user.

`localhost` → User can connect only from the local machine.

`Password@123` → Password for authentication.

This creates a new login account in MySQL.

4.2 Checking if User Exists

To verify existing users:

```
SELECT User, Host FROM mysql.user;
```

This shows:

- Username
- Host from which the user can connect

Example (Conceptual Table):

```
+---------------+-----------+
| User          | Host      |
+---------------+-----------+
| root          | localhost |
| college_user  | localhost |
+---------------+-----------+
```

If `college_user` appears, it means the user has been created successfully.

4.3 Granting Privileges to a User

Privileges define what actions a user can perform.

To grant full access on `College_DB`:

```
GRANT ALL PRIVILEGES ON College_DB.* TO
'college_user'@'localhost';
FLUSH PRIVILEGES;
```

Explanation:

- `ALL PRIVILEGES` → Allows all operations (SELECT, INSERT, UPDATE, DELETE, etc.).
- `College_DB.*` → Applies to all tables inside `College_DB`.
- `FLUSH PRIVILEGES` → Applies changes immediately.

4.4 Dropping an Existing User

To remove the user:

```
DROP USER 'college_user'@'localhost';
```

This deletes the user account and removes database access.

Steps, Code & Output:

```
mysql> -- Check existing databases
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.00 sec)

mysql> -- Create Database
mysql> CREATE DATABASE College_DB;
Query OK, 1 row affected (0.01 sec)

mysql> -- Use Database
mysql> USE College_DB;
Database changed
mysql> -- Drop Database
mysql> DROP DATABASE College_DB;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> -- Create User
mysql> CREATE USER 'college_user'@'localhost' IDENTIFIED BY 'Password@123';
Query OK, 0 rows affected (0.01 sec)

mysql> -- Check if user exists
mysql> SELECT User, Host FROM mysql.user WHERE User = 'college_user';
+--------------+-----------+
| User         | Host      |
+--------------+-----------+
| college_user | localhost |
+--------------+-----------+
1 row in set (0.00 sec)

mysql> -- Grant privileges
mysql> GRANT ALL PRIVILEGES ON College_DB.* TO 'college_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> -- Apply changes
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> -- Drop user
mysql> DROP USER 'college_user'@'localhost';
```

Learning Outcomes:

1. Set up a working MySQL database environment independently.
2. Execute SQL commands to manage databases effectively.
3. Perform user account management operations in MySQL.
4. Apply appropriate access control using privilege management.
5. Demonstrate confidence in handling basic administrative tasks in MySQL.

Course Outcomes:

This experiment contributes to the overall course by enabling students to understand the fundamental operations of database administration using MySQL. It strengthens their ability to install and configure database environments, create and manage databases, implement user management, and apply access control mechanisms. These skills form the foundation for advanced database design, security management, and real-world database system implementation, aligning with the course objective of developing practical competence in database management systems.

Conclusion:

This experiment helped in understanding the installation and basic setup of the MySQL environment using MySQL Workbench or XAMPP. It also demonstrated how to create, check, and drop databases as well as manage MySQL users and privileges. This forms the foundation for performing database operations and managing database security.

Viva Questions:

1. What is MySQL and how is it different from a database?
2. What is the purpose of the CREATE DATABASE command?
3. What does the SHOW DATABASES command do?
4. Why is user management important in MySQL?
5. What is the difference between creating a user and granting privileges to a user?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—2: Creating Tables with Constraints

Aim:

a) Create a table **students (id, name, age, department, email)** using appropriate data types.
b) Create a table **courses (course_id, course_name, duration, fees)** using appropriate data types.
c) Create a table **enrollments (enroll_id, student_id, course_id, enroll_date)** using appropriate data types.

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the concept of tables and their role in a relational database.
2. Identify and select appropriate data types for different attributes.
3. Apply constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE while creating tables.
4. Establish relationships between tables using foreign keys.
5. Design structured database tables based on real-world scenarios.

Theory:

1. Introduction to Tables in MySQL

In a relational database, data is stored in the form of **tables**.
A table consists of:

- **Rows (Records)** → Store actual data
- **Columns (Fields/Attributes)** → Define the type of data

Each table represents a real-world entity.

In this experiment:

- `students` represents student details
- `courses` represents course information
- `enrollments` represents the relationship between students and courses

2. Structure of a Table

A table is created using the `CREATE TABLE` statement.

Basic Syntax:

```
CREATE TABLE table_name (
    column1 datatype constraint,
    column2 datatype constraint,
    ...
);
```

Each column must have:

- A **name**
- A **data type**
- Optional **constraints**

3. Data Types in MySQL

Choosing the correct data type ensures efficient storage and data accuracy.

3.1 Common Data Types Used in This Practical

Data Type	Description	Example
INT	Stores whole numbers	101
VARCHAR(n)	Stores variable-length text	"Rahul"
DATE	Stores date values	2025-01-15
DECIMAL(p,s)	Stores numeric values with precision	15000.50

Explanation for This Practical:

- `id`, `course_id`, `enroll_id` → INT
- `name`, `department`, `course_name`, `email` → VARCHAR
- `age` → INT
- `duration` → INT (months)
- `fees` → DECIMAL
- `enroll_date` → DATE

4. Constraints in MySQL

Constraints are rules applied to columns to maintain **data integrity**.

They ensure:

- No duplicate values where not allowed.
- No null values where mandatory.
- Proper relationship between tables.

4.1 NOT NULL Constraint

Prevents a column from storing NULL values.

Example:

```
name VARCHAR(50) NOT NULL
```

Meaning: Every student must have a name.

4.2 PRIMARY KEY Constraint

- Uniquely identifies each record.
- Cannot contain NULL values.
- Only one primary key per table.

Example:

```
id INT PRIMARY KEY
```

4.3 UNIQUE Constraint

Ensures that all values in a column are different.

Example:

```
email VARCHAR(100) UNIQUE
```

Meaning: No two students can have the same email.

4.4 FOREIGN KEY Constraint

A foreign key creates a relationship between two tables.

It ensures that a value in one table must exist in another table.

Example Concept:

- `student_id` in `enrollments`
- Must exist as `id` in `students`

5. Relationship Between Tables

This practical involves three tables connected through relationships.

5.1 Entity Relationship Concept

```
+-----------+        +-------------+        +-------------+
|  students |        | enrollments |        |   courses   |
+-----------+        +-------------+        +-------------+
| id (PK)   |<----->| student_id  |        | course_id(PK)|
| name      |        | course_id   |<----->| course_name |
| age       |        | enroll_date |        | duration    |
| department|        +-------------+        | fees        |
| email     |
+-----------+
```

Explanation:

- `students.id` is Primary Key.
- `courses.course_id` is Primary Key.
- `enrollments.student_id` is a Foreign Key referencing `students.id`.
- `enrollments.course_id` is a Foreign Key referencing `courses.course_id`.

This creates a **many-to-many relationship** between students and courses using the enrollments table.

6. Purpose of Each Table

6.1 Students Table

Stores:

- Student ID
- Name
- Age
- Department
- Email

Constraints ensure:

- Each student has a unique ID.
- Email is unique.
- Mandatory fields cannot be null.

6.2 Courses Table

Stores:

- Course ID
- Course Name
- Duration
- Fees

Constraints ensure:

- Each course has a unique ID.
- Fees are properly stored using numeric data type.

6.3 Enrollments Table

Acts as a linking table.

Stores:

- Enrollment ID
- Student ID
- Course ID
- Enrollment Date

Foreign keys ensure:

- A student cannot enroll if the student does not exist.
- A course cannot be enrolled if it does not exist.

7. Importance of Constraints

Without constraints:

- Duplicate data may occur.
- Invalid references may be inserted.
- Data inconsistency may arise.

With constraints:

- Data accuracy is maintained.
- Relationships remain valid.
- Database becomes reliable and structured.

Code & Output:

(a)

```
mysql> -- ======================================
mysql> -- (a) Create students table
mysql> -- ======================================
mysql>
mysql> CREATE TABLE students (
    ->     id INT PRIMARY KEY AUTO_INCREMENT,
    ->     name VARCHAR(100) NOT NULL,
    ->     age INT,
    ->     department VARCHAR(100),
    ->     email VARCHAR(100) UNIQUE,
    ->     address VARCHAR(200),
    ->     category VARCHAR(50) DEFAULT 'Student'
    -> );
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> DESC students;
+------------+--------------+------+-----+---------+----------------+
| Field      | Type         | Null | Key | Default | Extra          |
+------------+--------------+------+-----+---------+----------------+
| id         | int          | NO   | PRI | NULL    | auto_increment |
| name       | varchar(100) | NO   |     | NULL    |                |
| age        | int          | YES  |     | NULL    |                |
| department | varchar(100) | YES  |     | NULL    |                |
| email      | varchar(100) | YES  | UNI | NULL    |                |
| address    | varchar(200) | YES  |     | NULL    |                |
| category   | varchar(50)  | YES  |     | Student |                |
+------------+--------------+------+-----+---------+----------------+
7 rows in set (0.01 sec)
```

(b)

```
mysql> -- ======================================
mysql> -- (b) Create courses table
mysql> -- ======================================
mysql>
mysql> CREATE TABLE courses (
    ->     course_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     course_name VARCHAR(100) NOT NULL,
    ->     duration INT,
    ->     fees DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DESC courses;
+-------------+---------------+------+-----+---------+----------------+
| Field       | Type          | Null | Key | Default | Extra          |
+-------------+---------------+------+-----+---------+----------------+
| course_id   | int           | NO   | PRI | NULL    | auto_increment |
| course_name | varchar(100)  | NO   |     | NULL    |                |
| duration    | int           | YES  |     | NULL    |                |
| fees        | decimal(10,2) | YES  |     | NULL    |                |
+-------------+---------------+------+-----+---------+----------------+
4 rows in set (0.00 sec)
```

निर्मलस्नेह उत्तम सेवाधर्म

(c)

```
mysql> -- ====================================
mysql> -- (c) Create enrollments table
mysql> -- ====================================
mysql>
mysql> CREATE TABLE enrollments (
    ->     enroll_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     student_id INT,
    ->     course_id INT,
    ->     enroll_date DATE,
    ->     FOREIGN KEY (student_id) REFERENCES students(id),
    ->     FOREIGN KEY (course_id) REFERENCES courses(course_id)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> DESC enrollments;
+-------------+------+------+-----+---------+----------------+
| Field       | Type | Null | Key | Default | Extra          |
+-------------+------+------+-----+---------+----------------+
| enroll_id   | int  | NO   | PRI | NULL    | auto_increment |
| student_id  | int  | YES  | MUL | NULL    |                |
| course_id   | int  | YES  | MUL | NULL    |                |
| enroll_date | date | YES  |     | NULL    |                |
+-------------+------+------+-----+---------+----------------+
4 rows in set (0.00 sec)
```

Learning Outcomes:

1. Create multiple tables using SQL with correct syntax and structure.
2. Implement data integrity using suitable constraints.
3. Define relationships between tables to maintain referential integrity.
4. Choose appropriate data types based on the nature of data.
5. Develop a basic relational database schema for a student-course enrollment system.

Course Outcomes:

This experiment supports the course objective of understanding relational database design and implementation by enabling students to create structured tables with appropriate data types and constraints. It enhances their ability to enforce data integrity using primary keys, foreign keys, and other constraints, and to establish relationships between multiple tables. These skills are essential for designing normalized databases and implementing real-world database systems with consistency, accuracy, and reliability.

Conclusion:

This experiment demonstrated how to create database tables using appropriate data types and constraints. The students table, courses table, and enrollments table were successfully created, helping to understand how relational data is structured and stored in a database.

Viva Questions:

1. What is the purpose of using constraints while creating a table in MySQL?
2. What is the difference between a PRIMARY KEY and a UNIQUE constraint?
3. Why is a FOREIGN KEY used in the enrollments table?
4. What happens if we try to insert a value in a foreign key column that does not exist in the referenced table?
5. Why is it important to choose appropriate data types while designing a table?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—3 : Altering and Dropping Tables

Aim:

a) Alter Table Structure - On table students (id, name, age, department, email).
Perform the following operations using ALTER TABLE:
- Add a new column phone of type VARCHAR(15).
- Modify the data type of age to SMALLINT.
- Drop the column department from the table.

b) Rename Table
Perform the following tasks:
- Rename the table courses to course_details.
- Rename the column fees to course_fees inside the same table.
- Add a new column category with default value 'General'.

c) Drop Table -
Perform the following operations:
- Drop the table enrollments from the database.
- Recreate a new table enrollments_new (enroll_id, student_id, course_id).
- Add a new column status with default 'Active' to enrollments_new.

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the purpose and syntax of the ALTER TABLE statement in MySQL.
2. Modify the structure of an existing table by adding, modifying, and dropping columns.
3. Rename tables and columns using appropriate SQL commands.
4. Understand the impact of structural changes on existing database objects.
5. Perform safe deletion and recreation of tables using DROP TABLE and CREATE TABLE.

Theory:

1. Introduction to Altering and Dropping Tables

In real-world database systems, requirements often change after tables are created. For example:

- A new field may need to be added.
- A column's data type may need modification.
- A column may no longer be required.
- A table name may need to be changed.
- An entire table may need to be deleted and recreated.

MySQL provides commands like:

- `ALTER TABLE`
- `RENAME`
- `DROP TABLE`

These commands allow modification of the database structure without recreating everything from scratch.

2. ALTER TABLE Statement

The `ALTER TABLE` statement is used to modify the structure of an existing table.

Basic Syntax:

```
ALTER TABLE table_name
operation;
```

Different operations can be performed using `ALTER TABLE`.

3. Adding a New Column

To add a new column to an existing table:

```
ALTER TABLE students
ADD phone VARCHAR(15);
```

Explanation:

- `ADD` is used to insert a new column.
- `phone` is the column name.
- `VARCHAR(15)` allows storing phone numbers up to 15 characters.

After adding, the table structure changes.

Before Adding:

```
students
---------------------------------
id | name | age | department | email
```

After Adding:

```
students
-------------------------------------------------
id | name | age | department | email | phone
```

4. Modifying a Column Data Type

To change the data type of an existing column:

```
ALTER TABLE students
MODIFY age SMALLINT;
```

Why Modify?

- `INT` takes more storage.
- `SMALLINT` is sufficient for storing age values.
- It optimizes storage and improves efficiency.

Important:
When modifying a column, existing data must be compatible with the new data type.

5. Dropping a Column

To remove a column permanently:

```
ALTER TABLE students
DROP department;
```

Effect:

- The column is deleted.
- All data stored in that column is permanently removed.
- This operation cannot be undone.

6. Renaming a Table

To rename a table:

```
RENAME TABLE courses TO course_details;
```

or

```
ALTER TABLE courses
RENAME TO course_details;
```

Purpose:

- Improve clarity.
- Match updated naming conventions.
- Reflect new system requirements.

7. Renaming a Column

To rename a column:

```
ALTER TABLE course_details
RENAME COLUMN fees TO course_fees;
```

Explanation:

- Changes only the column name.
- Does not affect stored data.
- Useful for better readability.

8. Adding a Column with Default Value

To add a new column with a default value:

```
ALTER TABLE course_details
ADD category VARCHAR(50) DEFAULT 'General';
```

What is Default Value?

A default value is automatically assigned when no value is provided during insertion.

Example:

If a new course is inserted without specifying `category`, it will automatically store:

```
General
```

9. Dropping a Table

To delete a table completely:

```
DROP TABLE enrollments;
```

Effect:

- Deletes table structure.
- Deletes all data inside it.
- Cannot be recovered unless backup exists.

10. Recreating a Table

After dropping, a new table can be created:

```
CREATE TABLE enrollments_new (
    enroll_id INT,
    student_id INT,
    course_id INT
);
```

This creates a fresh table with specified columns.

11. Adding Column with Default Value to New Table

```
ALTER TABLE enrollments_new
ADD status VARCHAR(20) DEFAULT 'Active';
```

Now the table structure becomes:

```
enrollments_new
-----------------------------------------
enroll_id | student_id | course_id | status
```

If no status is provided, it automatically stores:

```
Active
```

12. Importance of Table Modification Operations

These operations are essential because:

- Database requirements evolve over time.
- New attributes may be introduced.
- Old attributes may become unnecessary.
- Naming standards may change.
- Tables may need restructuring.

Using `ALTER TABLE`, `RENAME`, and `DROP TABLE` ensures flexibility in database design while maintaining control over structure.

Code & Output:

(a)

```
mysql> -- ====================================
mysql> -- (a) Alter students table
mysql> -- ====================================
mysql>
mysql> -- Add phone column
mysql> ALTER TABLE students
    -> ADD phone VARCHAR(15);
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Modify age datatype
mysql> ALTER TABLE students
    -> MODIFY age SMALLINT;
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Drop department column
mysql> ALTER TABLE students
    -> DROP COLUMN department;
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESC students;
+-------+--------------+------+-----+---------+----------------+
| Field | Type         | Null | Key | Default | Extra          |
+-------+--------------+------+-----+---------+----------------+
| id    | int          | NO   | PRI | NULL    | auto_increment |
| name  | varchar(100) | NO   |     | NULL    |                |
| age   | smallint     | YES  |     | NULL    |                |
| email | varchar(100) | YES  | UNI | NULL    |                |
| phone | varchar(15)  | YES  |     | NULL    |                |
+-------+--------------+------+-----+---------+----------------+
5 rows in set (0.01 sec)
```

(b)

```
mysql> -- ====================================
mysql> -- (b) Rename and modify courses table
mysql> -- ====================================
mysql>
mysql> -- Rename table courses to course_details
mysql> RENAME TABLE courses TO course_details;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> -- Rename column fees to course_fees
mysql> ALTER TABLE course_details
    -> CHANGE fees course_fees DECIMAL(10,2);
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Add new column category with default value 'General'
mysql> ALTER TABLE course_details
    -> ADD category VARCHAR(50) DEFAULT 'General';
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

```
mysql>
mysql> -- Verify
mysql> DESC course_details;
+-------------+---------------+------+-----+---------+----------------+
| Field       | Type          | Null | Key | Default | Extra          |
+-------------+---------------+------+-----+---------+----------------+
| course_id   | int           | NO   | PRI | NULL    | auto_increment |
| course_name | varchar(100)  | NO   |     | NULL    |                |
| duration    | int           | YES  |     | NULL    |                |
| course_fees | decimal(10,2) | YES  |     | NULL    |                |
| category    | varchar(50)   | YES  |     | General |                |
+-------------+---------------+------+-----+---------+----------------+
5 rows in set (0.01 sec)
```

(c)

```
mysql> -- ====================================
mysql> -- (c) Drop and recreate enrollments table
mysql> -- ====================================
mysql> USE College_DB;
Database changed
mysql>
mysql> -- Drop old table
mysql> DROP TABLE IF EXISTS enrollments;
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> -- Create new table
mysql> CREATE TABLE enrollments_new (
    ->     enroll_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     student_id INT,
    ->     course_id INT
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> -- Add status column with default value
mysql> ALTER TABLE enrollments_new
    -> ADD status VARCHAR(20) DEFAULT 'Active';
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Verify
mysql> DESC enrollments_new;
+------------+-------------+------+-----+---------+----------------+
| Field      | Type        | Null | Key | Default | Extra          |
+------------+-------------+------+-----+---------+----------------+
| enroll_id  | int         | NO   | PRI | NULL    | auto_increment |
| student_id | int         | YES  |     | NULL    |                |
| course_id  | int         | YES  |     | NULL    |                |
| status     | varchar(20) | YES  |     | Active  |                |
+------------+-------------+------+-----+---------+----------------+
4 rows in set (0.01 sec)
```

Learning Outcomes:

1. Alter existing database tables to accommodate new requirements.
2. Change data types of columns while maintaining data integrity.
3. Rename tables and columns without affecting database consistency.
4. Drop and recreate tables as per updated database design.
5. Demonstrate practical understanding of dynamic database schema modification.

Course Outcomes:

This experiment enhances the understanding of dynamic database schema modification by enabling students to alter, rename, drop, and recreate tables as per changing requirements. It strengthens their ability to manage database structures efficiently using SQL commands while maintaining data integrity and consistency. The skills developed in this experiment support the course objective of designing, modifying, and administering relational databases in real-world applications.

Conclusion:

This experiment provided practical knowledge of modifying existing table structures using the ALTER TABLE command. It also covered renaming tables and columns, dropping tables, and recreating tables. These operations are essential for maintaining and updating database structures.

Viva Questions:

1. What is the purpose of the ALTER TABLE command in MySQL?
2. What is the difference between MODIFY and CHANGE while altering a column?
3. What happens to the data when a column is dropped from a table?
4. Why is a default value used while adding a new column?
5. What is the difference between DROP TABLE and DELETE statement?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—4: Inserting Data into Tables

Aim:

a) Perform single line row insertion add 10 rows in Student table.
b) Perform multiline row insertion add 10 rows in enrollment tables, 5 rows in course.

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the purpose and syntax of the INSERT statement in MySQL.
2. Perform single-row insertion into a table using proper SQL syntax.
3. Perform multi-row insertion using a single INSERT statement.
4. Insert data while maintaining data type compatibility and constraints.
5. Populate related tables while maintaining referential integrity.

Theory:

1. Introduction to Data Insertion in MySQL

After creating tables, the next important step in database management is **inserting data** into those tables. Data insertion allows us to store actual records inside the database.

In MySQL, data is inserted using the `INSERT INTO` statement.

This experiment focuses on:

- Performing **single-row insertion** in the `students` table.
- Performing **multi-row insertion** in the `enrollments` and `courses` tables.

2. INSERT INTO Statement

The `INSERT INTO` statement is used to add new records into a table.

Basic Syntax:

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Important Points:

- Column names must match the table structure.
- Values must match the data type of the respective columns.
- If a column has constraints (e.g., PRIMARY KEY, NOT NULL), they must be satisfied.

3. Single-Line Row Insertion

Single-row insertion means inserting **one record at a time**.

3.1 Syntax

```
INSERT INTO students (id, name, age, email, phone)
VALUES (1, 'Rahul', 20, 'rahul@gmail.com', '9876543210');
```

3.2 Characteristics

- Each `INSERT` statement inserts only one row.
- Useful when inserting data manually.
- Easier to debug individual records.

3.3 Example Table Representation

Before Insertion:

```
students
----------------------------------
id | name | age | email | phone
---------------------------------
(empty)
```

After Inserting One Record:

```
students
------------------------------------------------------------
id | name  | age | email                | phone
------------------------------------------------------------
1  | Rahul | 20  | rahul@gmail.com      | 9876543210
```

To insert 10 rows using single-line insertion, 10 separate `INSERT` statements are executed.

4. Multi-Line Row Insertion

Multi-row insertion allows inserting **multiple records using a single INSERT statement**.

4.1 Syntax

```
INSERT INTO courses (course_id, course_name, duration,
course_fees)
VALUES
(101, 'DBMS', 6, 15000),
(102, 'Python', 4, 12000),
(103, 'Java', 6, 18000);
```

4.2 Characteristics

- More efficient than single-row insertion.
- Reduces query execution time.
- Useful when inserting bulk data.

5. Inserting Data into Related Tables

In relational databases, some tables are connected through **foreign keys**.

For example:

- `enrollments.student_id` references `students.id`
- `enrollments.course_id` references `course_details.course_id`

This means:

- A student must exist before inserting enrollment data.
- A course must exist before assigning it in enrollments.

If a non-existing student or course ID is inserted into `enrollments`, MySQL will generate a foreign key error.

6. Data Integrity During Insertion

While inserting data, the following rules must be followed:

1. **Primary Key Rule**
   - Cannot insert duplicate values.
   - Cannot insert NULL.
2. **NOT NULL Rule**
   - Column must have a value.
3. **Foreign Key Rule**
   - Value must exist in the referenced table.
4. **Data Type Matching**
   - Integer columns must receive numbers.
   - Date columns must follow `YYYY-MM-DD` format.
   - VARCHAR columns accept text.

7. Difference Between Single-Line and Multi-Line Insertion

Feature	Single-Line Insertion	Multi-Line Insertion
Number of Rows	One at a time	Multiple rows at once
Query Count	Multiple queries	Single query
Efficiency	Less efficient	More efficient
Usage	Manual entry	Bulk insertion

8. Importance of Data Insertion

Data insertion is important because:

- It populates the database with real data.
- It allows testing of relationships between tables.
- It helps verify constraints and foreign keys.
- It prepares the database for querying and analysis.

Code & Output:

(a)

```
mysql>
mysql> -- ====================================
mysql> -- (a) Single Line Insert - 10 Students
mysql> -- ====================================
mysql>
mysql> INSERT INTO students (name, age, email, address, category, phone)
    -> VALUES ('Krish Samanta', 20, 'krish@gmail.com', 'Mumbai', 'Student', '9876543210');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO students (name, age, email, address, category, phone)
    -> VALUES ('Riddhi Mestry', 19, 'riddhi@gmail.com', 'Mumbai', 'Student', '9876543211');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO students (name, age, email, address, category, phone)
    -> VALUES ('Jay Patil', 16, 'jay@gmail.com', 'Nashik', 'Student', '9876543212');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO students (name, age, email, address, category, phone)
    -> VALUES ('Vedant Gawade', 20, 'vedant@gmail.com', 'Thane', 'Student', '9876543213');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO students (name, age, email, address, category, phone)
    -> VALUES ('Ganesh Yadav', 18, 'ganesh@gmail.com', 'Mumbai', 'Student', '9876543214');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO students (name, age, email, address, category, phone)
    -> VALUES ('Ritesh Gupta', 21, 'ritesh@gmail.com', 'Delhi', 'Student', '9876543215');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO students (name, age, email, address, category, phone)
    -> VALUES ('Rishi Gupta', 22, 'rishi@gmail.com', 'Jaipur', 'Student', '9876543216');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO students (name, age, email, address, category, phone)
    -> VALUES ('Anuvesha Singh', 17, 'anuvesha@gmail.com', 'Lucknow', 'Student', '9876543217');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO students (name, age, email, address, category, phone)
    -> VALUES ('Sanchita Tiwari', 23, 'sanchita@gmail.com', 'Bhopal', 'Student', '9876543218');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO students (name, age, email, address, category, phone)
    -> VALUES ('Abhishek Khatri', 19, 'abhishek@gmail.com', 'Indore', 'Student', '9876543219');
Query OK, 1 row affected (0.00 sec)
```

(b)

```
mysql>
mysql> -- ======================================
mysql> -- (b) Multi-Line Insert - 5 Courses
mysql> -- ======================================
mysql>
mysql> INSERT INTO course_details (course_name, duration, course_fees, category)
    -> VALUES
    -> ('BSc CS', 36, 75000.00, 'Technical'),
    -> ('BCA', 36, 70000.00, 'Technical'),
    -> ('BSc IT', 36, 72000.00, 'Technical'),
    -> ('MSc CS', 24, 90000.00, 'Technical'),
    -> ('MSc IT', 24, 85000.00, 'Technical');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql>
mysql> -- ======================================
mysql> -- (b) Multi-Line Insert - Enrollments
mysql> -- ======================================
mysql>
mysql> INSERT INTO enrollments_new (student_id, course_id, status)
    -> VALUES
    -> (1, 1, 'Active'),
    -> (2, 1, 'Active'),
    -> (3, 3, 'Active'),
    -> (4, 4, 'Active'),
    -> (5, 5, 'Active'),
    -> (6, 1, 'Active'),
    -> (7, 2, 'Active'),
    -> (8, 3, 'Active'),
    -> (9, 4, 'Active'),
    -> (10, 5, 'Active');
Query OK, 10 rows affected (0.00 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
mysql>
mysql> -- ====================================
mysql> -- Display Output of All Tables
mysql> -- ====================================
mysql>
mysql> SELECT * FROM students;
+----+-----------------+------+--------------------+---------+----------+------------+
| id | name            | age  | email              | address | category | phone      |
+----+-----------------+------+--------------------+---------+----------+------------+
|  1 | Krish Samanta   |   20 | krish@gmail.com    | Mumbai  | Student  | 9876543210 |
|  2 | Riddhi Mestry   |   19 | riddhi@gmail.com   | Mumbai  | Student  | 9876543211 |
|  3 | Jay Patil       |   16 | jay@gmail.com      | Nashik  | Student  | 9876543212 |
|  4 | Vedant Gawade   |   20 | vedant@gmail.com   | Thane   | Student  | 9876543213 |
|  5 | Ganesh Yadav    |   18 | ganesh@gmail.com   | Mumbai  | Student  | 9876543214 |
|  6 | Ritesh Gupta    |   21 | ritesh@gmail.com   | Delhi   | Student  | 9876543215 |
|  7 | Rishi Gupta     |   22 | rishi@gmail.com    | Jaipur  | Student  | 9876543216 |
|  8 | Anuvesha Singh  |   17 | anuvesha@gmail.com | Lucknow | Student  | 9876543217 |
|  9 | Sanchita Tiwari |   23 | sanchita@gmail.com | Bhopal  | Student  | 9876543218 |
| 10 | Abhishek Khatri |   19 | abhishek@gmail.com | Indore  | Student  | 9876543219 |
+----+-----------------+------+--------------------+---------+----------+------------+
10 rows in set (0.00 sec)

mysql> SELECT * FROM course_details;
+-----------+-------------+----------+-------------+-----------+
| course_id | course_name | duration | course_fees | category  |
+-----------+-------------+----------+-------------+-----------+
|         1 | BSc CS      |       36 |    75000.00 | Technical |
|         2 | BCA         |       36 |    70000.00 | Technical |
|         3 | BSc IT      |       36 |    72000.00 | Technical |
|         4 | MSc CS      |       24 |    90000.00 | Technical |
|         5 | MSc IT      |       24 |    85000.00 | Technical |
+-----------+-------------+----------+-------------+-----------+
5 rows in set (0.00 sec)

mysql> SELECT * FROM enrollments_new;
+-----------+-----------+-----------+--------+
| enroll_id | student_id | course_id | status |
+-----------+-----------+-----------+--------+
|         1 |          1 |         1 | Active |
|         2 |          2 |         1 | Active |
|         3 |          3 |         3 | Active |
|         4 |          4 |         4 | Active |
|         5 |          5 |         5 | Active |
|         6 |          6 |         1 | Active |
|         7 |          7 |         2 | Active |
|         8 |          8 |         3 | Active |
|         9 |          9 |         4 | Active |
|        10 |         10 |         5 | Active |
+-----------+-----------+-----------+--------+
10 rows in set (0.00 sec)
```

Learning Outcomes:

1. Insert multiple records into database tables accurately.
2. Differentiate between single-line and multi-line insertion techniques.
3. Maintain consistency of data while inserting into related tables.
4. Handle insertion errors related to constraints and foreign keys.
5. Populate a relational database with meaningful sample data for testing and analysis.

Course Outcomes:

This experiment strengthens students' practical understanding of data manipulation in relational databases by enabling them to insert records using both single-row and multi-row insertion techniques. It enhances their ability to maintain data integrity while populating related tables and reinforces the application of constraints and referential integrity rules. These skills contribute to the overall course objective of developing competency in implementing and managing structured database systems using SQL.

Conclusion:

This experiment helped in understanding how to insert data into database tables using both single-row and multi-row insertion methods. It demonstrated how records are added to the students, enrollments, and courses tables efficiently.

Viva Questions:

1. What is the purpose of the INSERT INTO statement in MySQL?
2. What is the difference between single-row insertion and multi-row insertion?
3. What will happen if we try to insert a duplicate value into a PRIMARY KEY column?
4. Why must a referenced record exist before inserting data into a table with a FOREIGN KEY?
5. How can you insert values into a table without specifying column names, and when is it not recommended?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—5: DML Queries

Aim:

a) Select all columns from the students table
b) Select specific columns (name, email) from students
c) Update address of student whose roll no is 10
d) Delete student details whose category is student or delete student whose age is below 18

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the concept and purpose of Data Manipulation Language (DML) commands in SQL.
2. Retrieve data from a table using the SELECT statement.
3. Filter and display specific columns from a table.
4. Modify existing records using the UPDATE statement with conditions.
5. Remove records from a table using the DELETE statement based on specified criteria.

Theory:

1. Introduction to DML (Data Manipulation Language)

Data Manipulation Language (DML) consists of SQL commands used to **retrieve, modify, and delete data** stored in database tables.

Unlike DDL (which changes structure), DML works on the **data inside the tables**.

In this experiment, the following DML operations are performed on the `students` table:

- `SELECT` → Retrieve data
- `UPDATE` → Modify existing data
- `DELETE` → Remove data

2. SELECT Statement

The `SELECT` statement is used to retrieve data from a table.

2.1 Selecting All Columns

Syntax:

```
SELECT * FROM students;
```

Explanation:

- `*` means all columns.
- Retrieves complete records from the table.

Example Table:

```
students
-------------------------------------------------------
roll_no | name   | age | email            | category
-------------------------------------------------------
1       | Rahul  | 20  | r@gmail.com      | student
2       | Anu    | 17  | a@gmail.com      | student
```

Result of `SELECT *`:

All columns and all rows are displayed.

2.2 Selecting Specific Columns

Syntax:

```
SELECT name, email FROM students;
```

Explanation:

- Only selected columns are displayed.
- Useful when full data is not required.
- Improves performance for large tables.

Result:

```
----------------------------
name    | email
----------------------------
Rahul   | r@gmail.com
Anu     | a@gmail.com
```

3. WHERE Clause

The `WHERE` clause is used to filter records.

Syntax:

```
SELECT column_name
FROM table_name
WHERE condition;
```

Without `WHERE`, the query affects all records.

4. UPDATE Statement

The `UPDATE` statement modifies existing records in a table.

Syntax:

```
UPDATE students
SET address = 'Delhi'
WHERE roll_no = 10;
```

Explanation:

- `SET` specifies the column to change.
- `WHERE` ensures only the required record is updated.

Important Rule:

If `WHERE` is not used, all records in the table will be updated.

5. DELETE Statement

The `DELETE` statement removes records from a table.

5.1 Delete Based on Category

```
DELETE FROM students
WHERE category = 'student';
```

5.2 Delete Based on Age

```
DELETE FROM students
WHERE age < 18;
```

Explanation:

- Only records satisfying the condition are deleted.
- Data once deleted cannot be recovered without backup.

6. Difference Between UPDATE and DELETE

Feature	UPDATE	DELETE
Purpose	Modify existing data	Remove data
Removes Structure?	No	No
Removes Data?	No (modifies only)	Yes
Uses WHERE Clause?	Yes (recommended)	Yes (recommended)

7. Importance of DML Queries

DML queries are important because:

- They allow retrieval of stored data.
- They help correct mistakes using UPDATE.
- They help remove unwanted records.
- They support daily database operations in real-world systems.

Code & Output:

(a)

```
mysql> -- ====================================
mysql> -- (a) Select all columns from students
mysql> -- ====================================
mysql>
mysql> SELECT * FROM students;
+----+----------------+------+--------------------+---------+----------+------------+
| id | name           | age  | email              | address | category | phone      |
+----+----------------+------+--------------------+---------+----------+------------+
|  1 | Krish Samanta  |   20 | krish@gmail.com    | Mumbai  | Student  | 9876543210 |
|  2 | Riddhi Mestry  |   19 | riddhi@gmail.com   | Mumbai  | Student  | 9876543211 |
|  3 | Jay Patil      |   16 | jay@gmail.com      | Nashik  | Student  | 9876543212 |
|  4 | Vedant Gawade  |   20 | vedant@gmail.com   | Thane   | Student  | 9876543213 |
|  5 | Ganesh Yadav   |   18 | ganesh@gmail.com   | Mumbai  | Student  | 9876543214 |
|  6 | Ritesh Gupta   |   21 | ritesh@gmail.com   | Delhi   | Student  | 9876543215 |
|  7 | Rishi Gupta    |   22 | rishi@gmail.com    | Jaipur  | Student  | 9876543216 |
|  8 | Anuvesha Singh |   17 | anuvesha@gmail.com | Lucknow | Student  | 9876543217 |
|  9 | Sanchita Tiwari |  23 | sanchita@gmail.com | Bhopal  | Student  | 9876543218 |
| 10 | Abhishek Khatri |  19 | abhishek@gmail.com | Indore  | Student  | 9876543219 |
+----+----------------+------+--------------------+---------+----------+------------+
10 rows in set (0.00 sec)
```

(b)

```
mysql> -- ====================================
mysql> -- (b) Select specific columns (name, email)
mysql> -- ====================================
mysql>
mysql> SELECT name, email FROM students;
+----------------+--------------------+
| name           | email              |
+----------------+--------------------+
| Krish Samanta  | krish@gmail.com    |
| Riddhi Mestry  | riddhi@gmail.com   |
| Jay Patil      | jay@gmail.com      |
| Vedant Gawade  | vedant@gmail.com   |
| Ganesh Yadav   | ganesh@gmail.com   |
| Ritesh Gupta   | ritesh@gmail.com   |
| Rishi Gupta    | rishi@gmail.com    |
| Anuvesha Singh | anuvesha@gmail.com |
| Sanchita Tiwari | sanchita@gmail.com |
| Abhishek Khatri | abhishek@gmail.com |
+----------------+--------------------+
10 rows in set (0.00 sec)
```

(c)

```
mysql> -- Verify update
mysql> SELECT * FROM students WHERE id = 10;
+----+-----------------+------+--------------------+---------+----------+------------+
| id | name            | age  | email              | address | category | phone      |
+----+-----------------+------+--------------------+---------+----------+------------+
| 10 | Abhishek Khatri |   19 | abhishek@gmail.com | Chennai | Student  | 9876543219 |
+----+-----------------+------+--------------------+---------+----------+------------+
1 row in set (0.00 sec)
```

(d)

```
mysql> -- ====================================
mysql> -- (d) Delete student whose age is below 18
mysql> -- ====================================
mysql>
mysql> DELETE FROM students
    -> WHERE age < 18;
Query OK, 2 rows affected (0.00 sec)

mysql>
mysql> -- Verify deletion
mysql> SELECT * FROM students;
+----+-----------------+------+--------------------+---------+----------+------------+
| id | name            | age  | email              | address | category | phone      |
+----+-----------------+------+--------------------+---------+----------+------------+
|  1 | Krish Samanta   |   20 | krish@gmail.com    | Mumbai  | Student  | 9876543210 |
|  2 | Riddhi Mestry   |   19 | riddhi@gmail.com   | Mumbai  | Student  | 9876543211 |
|  4 | Vedant Gawade   |   20 | vedant@gmail.com   | Thane   | Student  | 9876543213 |
|  5 | Ganesh Yadav    |   18 | ganesh@gmail.com   | Mumbai  | Student  | 9876543214 |
|  6 | Ritesh Gupta    |   21 | ritesh@gmail.com   | Delhi   | Student  | 9876543215 |
|  7 | Rishi Gupta     |   22 | rishi@gmail.com    | Jaipur  | Student  | 9876543216 |
|  9 | Sanchita Tiwari |   23 | sanchita@gmail.com | Bhopal  | Student  | 9876543218 |
| 10 | Abhishek Khatri |   19 | abhishek@gmail.com | Chennai | Student  | 9876543219 |
+----+-----------------+------+--------------------+---------+----------+------------+
8 rows in set (0.00 sec)
```

Learning Outcomes:

1. Execute basic DML operations such as SELECT, UPDATE, and DELETE.
2. Retrieve complete or partial data from a database table efficiently.
3. Apply conditional statements using WHERE clause in queries.
4. Update specific records without affecting other data.
5. Safely delete records while maintaining data consistency.

Course Outcomes:

This experiment strengthens the understanding of Data Manipulation Language (DML) operations by enabling students to retrieve, update, and delete records from database tables using SQL. It enhances their ability to apply conditional statements effectively while maintaining data integrity and consistency. The practical skills gained contribute to the overall course objective of developing competency in querying and managing relational databases for real-world applications.

Conclusion:

This experiment introduced Data Manipulation Language (DML) operations such as SELECT, UPDATE, and DELETE. It helped in retrieving specific data, updating existing records, and removing unwanted records from the database.

Viva Questions:

1. What is the difference between DDL and DML commands in SQL?
2. What is the purpose of the WHERE clause in UPDATE and DELETE statements?
3. What will happen if the WHERE clause is omitted in an UPDATE statement?
4. How does SELECT * differ from selecting specific columns?
5. What is the difference between DELETE and TRUNCATE commands?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—6: Aggregate Functions

Aim:

a) Find the average age of students
b) Find the total fees collected for each course
c) Count how many students enrolled in each course
d) Count how many students are in each department

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the concept and purpose of aggregate functions in SQL.
2. Use aggregate functions such as AVG(), SUM(), and COUNT() to analyze data.
3. Apply the GROUP BY clause to categorize data for meaningful analysis.
4. Perform calculations on grouped data from multiple tables.
5. Interpret summarized results for decision-making and reporting purposes.

Theory:

1. Introduction to Aggregate Functions

Aggregate functions in SQL are used to perform calculations on a set of values and return a **single summarized result**.

Unlike normal queries that display multiple rows, aggregate functions produce **one result per group** or **one overall result**.

In this experiment, we use aggregate functions to:

- Find the average age of students
- Calculate total fees collected per course
- Count students enrolled in each course
- Count students in each department

2. Common Aggregate Functions in MySQL

2.1 AVG() – Average Function

The `AVG()` function calculates the average value of a numeric column.

Syntax:

```
SELECT AVG(column_name)
FROM table_name;
```

Example Use Case:

To find the average age of students.

Important Points:

- Works only on numeric columns.
- Ignores NULL values.
- Returns a single value.

2.2 SUM() – Total Function

The `SUM()` function calculates the total of a numeric column.

Syntax:

```
SELECT SUM(column_name)
FROM table_name;
```

Example Use Case:

To find the total fees collected for each course.

2.3 COUNT() – Counting Records

The `COUNT()` function returns the number of rows.

Syntax:

```
SELECT COUNT(column_name)
FROM table_name;
```

Two Common Forms:

- `COUNT(*)` → Counts all rows
- `COUNT(column_name)` → Counts non-NULL values

Example Use Case:

- Counting how many students enrolled in each course
- Counting how many students are in each department

3. GROUP BY Clause

When we need results **for each category**, we use the `GROUP BY` clause.

Without `GROUP BY`, aggregate functions calculate a result for the entire table.

With `GROUP BY`, results are calculated for each group separately.

Syntax:

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

4. Applying Aggregate Functions in This Experiment

4.1 Average Age of Students

- Uses `AVG(age)`
- Returns a single value
- No grouping required

Concept Example:

If ages are:

```
18, 20, 22
```

Average = (18 + 20 + 22) / 3 = 20

4.2 Total Fees Collected for Each Course

To calculate total fees per course:

- Use `SUM(fees)`
- Use `GROUP BY course_id`

This groups all enrollments by course and sums the fees collected.

Concept Representation:

```
Course A → 3 students × 15000 = 45000
Course B → 2 students × 12000 = 24000
```

4.3 Count Students Enrolled in Each Course

- Use `COUNT(student_id)`
- Group by `course_id`

Concept Representation:

```
Course 101 → 5 students
Course 102 → 3 students
```

4.4 Count Students in Each Department

- Use `COUNT(*)`
- Group by `department`

Concept Representation:

```
CSE → 10 students
ECE → 8 students
ME → 5 students
```

5. Difference Between Aggregate Without GROUP BY and With GROUP BY

Without GROUP BY	With GROUP BY
One overall result	One result per group
Entire table considered	Data divided into categories
Example: Average age of all students	Example: Count students per department

6. Importance of Aggregate Functions

Aggregate functions are important because they:

- Summarize large amounts of data
- Help in reporting and analysis
- Support decision-making
- Provide statistical insights
- Reduce data into meaningful information

Code & Output:

(a)

```
mysql> -- ====================================
mysql> -- (a) Find the average age of students
mysql> -- ====================================
mysql>
mysql> SELECT AVG(age) AS Average_Age
    -> FROM students;
+-------------+
| Average_Age |
+-------------+
|     20.2500 |
+-------------+
1 row in set (0.01 sec)
```

(b)

```
mysql> -- ====================================
mysql> -- (b) Find the total fees of each course
mysql> -- ====================================
mysql>
mysql> SELECT course_name,
    ->        SUM(course_fees) AS Total_Fees
    -> FROM course_details
    -> GROUP BY course_name;
+-------------+------------+
| course_name | Total_Fees |
+-------------+------------+
| BSc CS      |   75000.00 |
| BCA         |   70000.00 |
| BSc IT      |   72000.00 |
| MSc CS      |   90000.00 |
| MSc IT      |   85000.00 |
+-------------+------------+
5 rows in set (0.00 sec)
```

(c)

```
mysql> -- ====================================
mysql> -- (c) Count how many students enrolled in each course
mysql> -- ====================================
mysql>
mysql> SELECT course_id,
    ->        COUNT(student_id) AS Total_Students
    -> FROM enrollments_new
    -> GROUP BY course_id;
+-----------+----------------+
| course_id | Total_Students |
+-----------+----------------+
|         1 |              3 |
|         3 |              2 |
|         4 |              2 |
|         5 |              2 |
|         2 |              1 |
+-----------+----------------+
5 rows in set (0.00 sec)
```

(d)

```
mysql>
mysql> -- Add department column
mysql> ALTER TABLE students
    -> ADD department VARCHAR(100);
Query OK, 0 rows affected (0.01 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Update department values
mysql>
mysql> UPDATE students
    -> SET department = 'Computer Science'
    -> WHERE id IN (1,2,4,6,9);
Query OK, 5 rows affected (0.00 sec)
Rows matched: 5  Changed: 5  Warnings: 0

mysql>
mysql> UPDATE students
    -> SET department = 'Information Technology'
    -> WHERE id IN (5,10);
Query OK, 2 rows affected (0.00 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql>
mysql> UPDATE students
    -> SET department = 'Computer Applications'
    -> WHERE id = 7;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql>
mysql> -- ====================================
mysql> -- (d) Count how many students are in each department
mysql> -- ====================================
mysql>
mysql> SELECT department,
    ->        COUNT(*) AS Total_Students
    -> FROM students
    -> GROUP BY department;
+-----------------------+----------------+
| department            | Total_Students |
+-----------------------+----------------+
| Computer Science      |              5 |
| Information Technology |              2 |
| Computer Applications |              1 |
+-----------------------+----------------+
3 rows in set (0.00 sec)
```

Learning Outcomes:

1. Calculate average values, totals, and counts using SQL aggregate functions.
2. Group records based on specific columns using the GROUP BY clause.
3. Analyze student and course data to generate summarized reports.
4. Perform aggregate queries across related tables accurately.
5. Apply data aggregation techniques in real-world database applications.

Course Outcomes:

This experiment enhances students' ability to analyze and summarize data using SQL aggregate functions such as AVG(), SUM(), and COUNT() along with the GROUP BY clause. It strengthens their understanding of data analysis within relational databases and enables them to generate meaningful reports from stored data. The skills developed contribute to the overall course objective of applying SQL techniques for effective data retrieval, analysis, and decision-making in real-world database applications.

Conclusion:

This experiment demonstrated the use of aggregate functions such as AVG(), SUM(), and COUNT() to perform calculations on stored data. These functions help summarize large datasets and generate useful statistical information.

Viva Questions:

1. What are aggregate functions in SQL? Give examples.
2. What is the difference between COUNT(*) and COUNT(column_name)?
3. Why is the GROUP BY clause used with aggregate functions?
4. What will happen if we use an aggregate function without GROUP BY?
5. Can aggregate functions be used on non-numeric columns? If yes, which one and how?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—7: Conditional Queries

Aim:

a) Students older than 18 AND from 'Computer' department
b) Students whose name starts with 'A'
c) Courses with duration BETWEEN 3 and 12 months
d) Enrollments where course_id = 2 OR student_id = 5

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the use of conditional operators in SQL queries.
2. Apply logical operators such as AND, OR, and comparison operators effectively.
3. Use pattern matching with the LIKE operator.
4. Filter records using range conditions with the BETWEEN operator.
5. Retrieve specific records from tables based on multiple conditions.

Theory:

1. Introduction to Conditional Queries

Conditional queries in SQL are used to retrieve records based on specific conditions. They help in filtering data from tables using logical and comparison operators.

In this experiment, conditions are applied to:

- Filter students based on age and department
- Find students whose names follow a pattern
- Retrieve courses within a duration range
- Filter enrollments using multiple conditions

All these operations are performed using the `WHERE` clause.

2. WHERE Clause

The `WHERE` clause is used to specify conditions while retrieving data.

Basic Syntax:

```
SELECT column_name
FROM table_name
WHERE condition;
```

Without the `WHERE` clause, all records are returned.

3. Logical Operators

Logical operators are used to combine multiple conditions.

3.1 AND Operator

The `AND` operator returns records only if **all conditions are true**.

Syntax:

```
SELECT *
FROM students
WHERE age > 18 AND department = 'Computer';
```

Explanation:

- Students must be older than 18.
- Students must belong to the 'Computer' department.
- Both conditions must be satisfied.

Concept Representation:

```
Condition 1: age > 18  → TRUE
Condition 2: department = 'Computer' → TRUE
Result → Record Displayed
```

If either condition is false, the record is not displayed.

3.2 OR Operator

The `OR` operator returns records if **at least one condition is true**.

Syntax:

```
SELECT *
FROM enrollments
WHERE course_id = 2 OR student_id = 5;
```

Explanation:

- Displays enrollments where:
  - Course ID is 2
    OR
  - Student ID is 5

If any one condition matches, the record is selected.

4. Pattern Matching Using LIKE

The `LIKE` operator is used for searching text patterns.

It works with wildcard characters:

- `%` → Represents zero or more characters
- `_` → Represents exactly one character

Example: Names Starting with 'A'

```
SELECT *
FROM students
WHERE name LIKE 'A%';
```

Explanation:

- `A%` means:
  - Name starts with letter 'A'
  - Followed by any number of characters

Example Matches:

```
Amit
Anu
Akash
```

5. Range Filtering Using BETWEEN

The `BETWEEN` operator selects values within a specified range.

Syntax:

```
SELECT *
FROM courses
WHERE duration BETWEEN 3 AND 12;
```

Explanation:

- Selects courses with duration from 3 to 12 months.
- Includes both 3 and 12 (inclusive).

Equivalent to:

```
WHERE duration >= 3 AND duration <= 12;
```

6. Comparison Operators Used

Operator	Meaning
=	Equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
BETWEEN	Within range
LIKE	Pattern matching

7. Importance of Conditional Queries

Conditional queries are important because they:

- Retrieve only required data.
- Improve efficiency by reducing unnecessary output.
- Help in searching and filtering large datasets.
- Support decision-making and reporting.
- Enable complex data retrieval using logical combinations.

Code & Output:

(a)

```
mysql>
mysql> -- ====================================
mysql> -- (a) Students older than 18 AND from 'Computer Science' department
mysql> -- ====================================
mysql>
mysql> SELECT *
    -> FROM students
    -> WHERE age > 18
    -> AND department = 'Computer Science';
+----+-----------------+------+--------------------+---------+----------+------------+------------------+
| id | name            | age  | email              | address | category | phone      | department       |
+----+-----------------+------+--------------------+---------+----------+------------+------------------+
|  1 | Krish Samanta   |   20 | krish@gmail.com    | Mumbai  | Student  | 9876543210 | Computer Science |
|  2 | Riddhi Mestry   |   19 | riddhi@gmail.com   | Mumbai  | Student  | 9876543211 | Computer Science |
|  4 | Vedant Gawade   |   20 | vedant@gmail.com   | Thane   | Student  | 9876543213 | Computer Science |
|  6 | Ritesh Gupta    |   21 | ritesh@gmail.com   | Delhi   | Student  | 9876543215 | Computer Science |
|  9 | Sanchita Tiwari |   23 | sanchita@gmail.com | Bhopal  | Student  | 9876543218 | Computer Science |
+----+-----------------+------+--------------------+---------+----------+------------+------------------+
5 rows in set (0.00 sec)
```

(b)

```
mysql> -- ====================================
mysql> -- (b) Students whose name starts with 'A'
mysql> -- ====================================
mysql>
mysql> SELECT *
    -> FROM students
    -> WHERE name LIKE 'A%';
+----+-----------------+------+--------------------+---------+----------+------------+------------------------+
| id | name            | age  | email              | address | category | phone      | department             |
+----+-----------------+------+--------------------+---------+----------+------------+------------------------+
| 10 | Abhishek Khatri |   19 | abhishek@gmail.com | Chennai | Student  | 9876543219 | Information Technology |
+----+-----------------+------+--------------------+---------+----------+------------+------------------------+
1 row in set (0.01 sec)
```

(c)

```
mysql> -- Update one course duration to satisfy condition
mysql> UPDATE course_details
    -> SET duration = 6
    -> WHERE course_name = 'BCA';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> SELECT * FROM course_details;
+-----------+-------------+----------+-------------+-----------+
| course_id | course_name | duration | course_fees | category  |
+-----------+-------------+----------+-------------+-----------+
|         1 | BSc CS      |       36 |    75000.00 | Technical |
|         2 | BCA         |        6 |    70000.00 | Technical |
|         3 | BSc IT      |       36 |    72000.00 | Technical |
|         4 | MSc CS      |       24 |    90000.00 | Technical |
|         5 | MSc IT      |       24 |    85000.00 | Technical |
+-----------+-------------+----------+-------------+-----------+
5 rows in set (0.00 sec)
```

```
mysql> -- ====================================
mysql> -- (c) Courses with duration BETWEEN 3 and 12 months
mysql> -- ====================================
mysql>
mysql> SELECT *
    -> FROM course_details
    -> WHERE duration BETWEEN 3 AND 12;
+-----------+-------------+----------+-------------+-----------+
| course_id | course_name | duration | course_fees | category  |
+-----------+-------------+----------+-------------+-----------+
|         2 | BCA         |        6 |    70000.00 | Technical |
+-----------+-------------+----------+-------------+-----------+
1 row in set (0.00 sec)
```

(d)

```
mysql> -- ====================================
mysql> -- (d) Enrollments where course_id = 2 OR student_id = 5
mysql> -- ====================================
mysql>
mysql> SELECT *
    -> FROM enrollments_new
    -> WHERE course_id = 2
    -> OR student_id = 5;
+-----------+------------+-----------+--------+
| enroll_id | student_id | course_id | status |
+-----------+------------+-----------+--------+
|         5 |          5 |         5 | Active |
|         7 |          7 |         2 | Active |
+-----------+------------+-----------+--------+
2 rows in set (0.00 sec)
```

Learning Outcomes:

1. Write SQL queries using conditional clauses to filter data accurately.
2. Combine multiple conditions using logical operators.
3. Perform pattern-based searches in text fields.
4. Apply range-based filtering using the BETWEEN clause.
5. Construct efficient queries to extract meaningful data from relational tables.

Course Outcomes:

This experiment enhances students' ability to construct conditional SQL queries using logical operators, comparison operators, pattern matching, and range filtering. It strengthens their understanding of data retrieval techniques by enabling them to apply multiple conditions effectively using the WHERE clause. The skills developed contribute to the overall course objective of designing efficient queries for accurate data extraction and analysis in relational database systems.

Conclusion:

This experiment helped in understanding how to filter records using conditional operators such as AND, OR, LIKE, and BETWEEN. It showed how SQL queries can retrieve specific data based on defined conditions.

Viva Questions:

1. What is the purpose of the WHERE clause in SQL?
2. What is the difference between AND and OR operators?
3. What wildcard characters are used with the LIKE operator?
4. Is the BETWEEN operator inclusive or exclusive? Explain.
5. How is BETWEEN different from using >= and <= conditions?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—8 : Group By and Having Clause

Aim:

For Tables:
Students(name, age, email, phone_no, category)
Enrollments(enroll_id, Student_id, course_id, enroll_date)
course_details(cname, duration, FEES_DETAILS)

a) Write a query to find the total number of students in each category from the Students table.
b) Write a query to find the number of students in each age group from the Students table.
c) Display categories having more than 3 students
d) Find courses having more than 5 enrollments

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the purpose and functionality of the GROUP BY clause in SQL.
2. Apply aggregate functions such as COUNT() along with grouping.
3. Differentiate between WHERE and HAVING clauses.
4. Filter grouped data using the HAVING clause.
5. Analyze categorized and summarized data effectively.

Theory:

1. Introduction to GROUP BY Clause

In SQL, the `GROUP BY` clause is used to arrange identical data into groups. It is commonly used with **aggregate functions** such as:

- `COUNT()`
- `SUM()`
- `AVG()`
- `MAX()`
- `MIN()`

Instead of showing individual rows, `GROUP BY` helps in summarizing data based on specific columns.

In this experiment, grouping is performed on:

- Category of students
- Age groups
- Courses
- Enrollment records

2. Syntax of GROUP BY

Basic Syntax:

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

Important Rules:

1. Columns in the SELECT statement must either:
   - Be included in the `GROUP BY` clause
     OR
   - Be used inside an aggregate function.
2. `GROUP BY` is written after the `FROM` clause.
3. `GROUP BY` is used before `HAVING`.

3. Counting Students in Each Category

To count how many students belong to each category, we use:

- `COUNT(*)`
- `GROUP BY category`

Conceptual Representation:

Students Table:

```
name    | category
-------------------
Amit    | Regular
Anu     | Regular
Riya    | Diploma
Kiran   | Regular
```

After applying GROUP BY:

```
category | total_students
---------------------------
Regular  | 3
Diploma  | 1
```

Each category forms a separate group.

4. Grouping by Age

When grouping by age:

- Students having the same age are grouped together.
- `COUNT(*)` calculates how many students fall under each age.

Conceptual Example:

```
age | total_students
---------------------
18  | 4
19  | 6
20  | 3
```

This helps analyze student distribution by age.

5. HAVING Clause

The `HAVING` clause is used to filter grouped results.

Difference between WHERE and HAVING:

WHERE	HAVING
Filters rows before grouping	Filters groups after grouping
Cannot use aggregate functions	Can use aggregate functions
Applied on individual records	Applied on grouped data

Syntax of HAVING

```
SELECT column_name, COUNT(*)
FROM table_name
GROUP BY column_name
HAVING COUNT(*) > 3;
```

Explanation:

- First, rows are grouped.
- Then, the `HAVING` condition filters groups.

6. Display Categories Having More Than 3 Students

Process:

1. Group students by category.
2. Count number of students in each category.
3. Apply `HAVING COUNT(*) > 3`.

This ensures only categories with more than 3 students are displayed.

7. Find Courses Having More Than 5 Enrollments

To determine courses with high enrollment:

1. Group records in `Enrollments` table by `course_id`.
2. Count number of students enrolled in each course.
3. Use `HAVING COUNT(*) > 5` to filter.

Conceptual Example:

```
course_id | total_enrollments
--------------------------------
1         | 6
2         | 3
3         | 8
```

After applying HAVING:

```
course_id | total_enrollments
--------------------------------
1         | 6
3         | 8
```

Courses with more than 5 enrollments are displayed.

8. Order of Execution in GROUP BY Queries

The execution order of a grouped query:

```
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
```

Understanding this order helps avoid logical errors in query writing.

9. Importance of GROUP BY and HAVING

These clauses are important because they:

- Summarize large datasets.
- Generate category-wise or group-wise reports.
- Help in statistical analysis.
- Support decision-making processes.
- Allow filtering based on aggregated results.

Code & Output:

(a)

```
mysql> -- ====================================
mysql> -- (a) Total number of students in each category
mysql> -- ====================================
mysql>
mysql> SELECT category,
    ->        COUNT(*) AS Total_Students
    -> FROM students
    -> GROUP BY category;
+----------+----------------+
| category | Total_Students |
+----------+----------------+
| Student  |              8 |
+----------+----------------+
1 row in set (0.00 sec)
```

(b)

```
mysql> -- ====================================
mysql> -- (b) Number of students in each age group
mysql> -- ====================================
mysql>
mysql> SELECT age,
    ->        COUNT(*) AS Total_Students
    -> FROM students
    -> GROUP BY age;
+------+----------------+
| age  | Total_Students |
+------+----------------+
|   20 |              2 |
|   19 |              2 |
|   18 |              1 |
|   21 |              1 |
|   22 |              1 |
|   23 |              1 |
+------+----------------+
6 rows in set (0.00 sec)
```

(c)

```
mysql> -- ====================================
mysql> -- (c) Display categories having more than 3 students
mysql> -- ====================================
mysql>
mysql> SELECT category,
    ->        COUNT(*) AS Total_Students
    -> FROM students
    -> GROUP BY category
    -> HAVING COUNT(*) > 3;
+----------+----------------+
| category | Total_Students |
+----------+----------------+
| Student  |              8 |
+----------+----------------+
1 row in set (0.01 sec)
```

(d)

```
mysql> -- ====================================
mysql> -- (d) Find courses having more than 5 enrollments
mysql> -- ====================================
mysql>
mysql> SELECT course_id,
    ->        COUNT(student_id) AS Total_Enrollments
    -> FROM enrollments_new
    -> GROUP BY course_id
    -> HAVING COUNT(student_id) > 5;
Empty set (0.00 sec)
```

Learning Outcomes:

1. Group records based on specific columns to generate summarized results.
2. Use aggregate functions with GROUP BY to analyze data patterns.
3. Apply the HAVING clause to filter aggregated results.
4. Generate meaningful reports such as category-wise and course-wise summaries.
5. Construct advanced SQL queries for grouped data analysis in relational databases

Course Outcomes:

This experiment enhances students' ability to analyze and summarize grouped data using the GROUP BY and HAVING clauses in SQL. It strengthens their understanding of applying aggregate functions to categorized datasets and filtering aggregated results effectively. The practical knowledge gained supports the overall course objective of developing advanced querying skills for data analysis, reporting, and decision-making in relational database systems.

Conclusion:

This experiment demonstrated how to group data using the GROUP BY clause and filter grouped results using the HAVING clause. It helped in analyzing categorized data and generating summarized reports.

Viva Questions:

1. What is the purpose of the GROUP BY clause in SQL?
2. What is the difference between WHERE and HAVING clauses?
3. Can we use aggregate functions in the WHERE clause? Why or why not?
4. What will happen if a column is selected but not included in the GROUP BY clause?
5. Explain the execution order of a SQL query containing GROUP BY and HAVING.

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—9: Keys and Constrains

Aim:

Implement PRIMARY KEY, FOREIGN KEY ,UNIQUE, NOT NULL, CHECK, DEFAULT ,CASCADE / SET NULL on

Customers(Customer_ID, Customer_name- not Null, Phone_No- Unique, Gender-check),
Categories(Category_id - auto_increment, category_name-Unique,not null)
Products(Id, name-not null, category_id, price, stock_quantity, expiry_date)

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the concept and purpose of keys and constraints in relational databases.
2. Implement PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT constraints.
3. Apply AUTO_INCREMENT to generate unique values automatically.
4. Establish relationships between tables using FOREIGN KEY constraints.
5. Use referential actions such as CASCADE and SET NULL to maintain data integrity.

Theory:

1. Introduction to Keys and Constraints

In relational databases, **keys and constraints** are used to maintain **data integrity, accuracy, and consistency**.

They ensure that:

- Data is unique where required.
- Mandatory fields are not left empty.
- Relationships between tables are properly maintained.
- Invalid data is restricted.

In this experiment, constraints are implemented on three tables:

- `Customers`
- `Categories`
- `Products`

2. Types of Keys

2.1 Primary Key

A **PRIMARY KEY** uniquely identifies each record in a table.

Properties:

- Cannot contain NULL values.
- Must be unique.
- Only one primary key per table.

Example:

```
Customer_ID INT PRIMARY KEY
```

This ensures that every customer has a unique ID.

2.2 Foreign Key

A **FOREIGN KEY** creates a relationship between two tables.

It ensures that:

- A value in one table must exist in another table.

Example:

```
category_id INT,
FOREIGN KEY (category_id) REFERENCES Categories(Category_id)
```

This ensures that a product must belong to a valid category.

3. Constraints

Constraints are rules applied to columns to restrict the type of data that can be inserted.

3.1 NOT NULL Constraint

Ensures that a column cannot have NULL values.

Example:

```
Customer_name VARCHAR(50) NOT NULL
```

This means every customer must have a name.

3.2 UNIQUE Constraint

Ensures that all values in a column are different.

Example:

```
Phone_No VARCHAR(15) UNIQUE
```

This prevents duplicate phone numbers.

3.3 CHECK Constraint

Ensures that values satisfy a specific condition.

Example:

```
Gender VARCHAR(10) CHECK (Gender IN ('Male','Female'))
```

This ensures only valid gender values are allowed.

3.4 DEFAULT Constraint

Assigns a default value if no value is provided during insertion.

Example:

```
stock_quantity INT DEFAULT 0
```

If no stock value is provided, it automatically stores 0.

3.5 AUTO_INCREMENT

Used to automatically generate sequential numbers.

Example:

```
Category_id INT AUTO_INCREMENT PRIMARY KEY
```

This automatically generates category IDs.

4. Referential Actions

When a foreign key relationship exists, we can define actions that occur when a parent record is updated or deleted.

4.1 CASCADE

If a parent record is deleted or updated, related records in the child table are automatically deleted or updated.

Example:

```
FOREIGN KEY (category_id)
REFERENCES Categories(Category_id)
ON DELETE CASCADE
```

If a category is deleted, all related products are also deleted.

4.2 SET NULL

If a parent record is deleted, the foreign key in the child table is set to NULL.

Example:

```
FOREIGN KEY (category_id)
REFERENCES Categories(Category_id)
ON DELETE SET NULL
```

If a category is deleted, the `category_id` in Products becomes NULL.

5. Table Relationships in This Experiment

Conceptual Relationship Diagram:

```
Customers
----------
Customer_ID (PK)
Customer_name (NOT NULL)
Phone_No (UNIQUE)
Gender (CHECK)

Categories
----------
Category_id (PK, AUTO_INCREMENT)
Category_name (UNIQUE, NOT NULL)

Products
----------
Id (PK)
name (NOT NULL)
category_id (FK)
price
stock_quantity (DEFAULT 0)
expiry_date
```

Relationship:

```
Categories (1) -------- (Many) Products
```

One category can have many products.

6. Importance of Keys and Constraints

Keys and constraints are important because:

- They prevent duplicate and invalid data.
- They enforce business rules.
- They maintain relationships between tables.
- They improve database reliability.
- They ensure data consistency across related tables.

Code & Output:

```
mysql> -- Create and use new database
mysql> CREATE DATABASE Store_DB;
Query OK, 1 row affected (0.01 sec)

mysql> USE Store_DB;
Database changed
mysql> -- ====================================
mysql> -- Create Customers table
mysql> -- ====================================
mysql>
mysql> CREATE TABLE Customers (
    ->     Customer_ID INT PRIMARY KEY,
    ->     Customer_name VARCHAR(100) NOT NULL,
    ->     Phone_No VARCHAR(15) UNIQUE,
    ->     Gender VARCHAR(10),
    ->     CHECK (Gender IN ('Male', 'Female', 'Other'))
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> -- ====================================
mysql> -- Create Categories table
mysql> -- ====================================
mysql>
mysql> CREATE TABLE Categories (
    ->     Category_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     category_name VARCHAR(100) UNIQUE NOT NULL
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> -- ====================================
mysql> -- Create Products table
mysql> -- ====================================
mysql>
mysql> CREATE TABLE Products (
    ->     Id INT PRIMARY KEY,
    ->     name VARCHAR(100) NOT NULL,
    ->     category_id INT,
    ->     price DECIMAL(10,2) CHECK (price > 0),
    ->     stock_quantity INT DEFAULT 0,
    ->     expiry_date DATE,
    ->
    ->     FOREIGN KEY (category_id)
    ->     REFERENCES Categories(Category_id)
    ->     ON DELETE SET NULL
    ->     ON UPDATE CASCADE
    -> );
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> -- ====================================
mysql> -- Show Table Structure
mysql> -- ====================================
mysql>
mysql> DESC Customers;
+---------------+--------------+------+-----+---------+-------+
| Field         | Type         | Null | Key | Default | Extra |
+---------------+--------------+------+-----+---------+-------+
| Customer_ID   | int          | NO   | PRI | NULL    |       |
| Customer_name | varchar(100) | NO   |     | NULL    |       |
| Phone_No      | varchar(15)  | YES  | UNI | NULL    |       |
| Gender        | varchar(10)  | YES  |     | NULL    |       |
+---------------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)

mysql> DESC Categories;
+---------------+--------------+------+-----+---------+----------------+
| Field         | Type         | Null | Key | Default | Extra          |
+---------------+--------------+------+-----+---------+----------------+
| Category_id   | int          | NO   | PRI | NULL    | auto_increment |
| category_name | varchar(100) | NO   | UNI | NULL    |                |
+---------------+--------------+------+-----+---------+----------------+
2 rows in set (0.00 sec)

mysql> DESC Products;
+----------------+---------------+------+-----+---------+-------+
| Field          | Type          | Null | Key | Default | Extra |
+----------------+---------------+------+-----+---------+-------+
| Id             | int           | NO   | PRI | NULL    |       |
| name           | varchar(100)  | NO   |     | NULL    |       |
| category_id    | int           | YES  | MUL | NULL    |       |
| price          | decimal(10,2) | YES  |     | NULL    |       |
| stock_quantity | int           | YES  |     | 0       |       |
| expiry_date    | date          | YES  |     | NULL    |       |
+----------------+---------------+------+-----+---------+-------+
6 rows in set (0.00 sec)
```

Learning Outcomes:

1. Design tables with appropriate keys and constraints to ensure data integrity.
2. Enforce uniqueness, mandatory fields, and validation rules in database tables.
3. Create relationships between tables using foreign keys effectively.
4. Implement referential integrity using CASCADE and SET NULL options.
5. Develop structured and secure relational database schemas for real-world applications.

Course Outcomes:

This experiment strengthens students' understanding of implementing keys and constraints to ensure data integrity and consistency in relational databases. It enables them to design structured tables using PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT, and AUTO_INCREMENT constraints, and to maintain referential integrity using CASCADE and SET NULL options. The practical knowledge gained supports the overall course objective of developing robust, secure, and well-structured database schemas suitable for real-world applications.

Conclusion:

This experiment provided practical understanding of implementing keys and constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT. These constraints ensure data integrity and maintain relationships between database tables.

Viva Questions:

1. What is the difference between a PRIMARY KEY and a UNIQUE constraint?
2. Why is a FOREIGN KEY used in a relational database?
3. What is the purpose of the CHECK constraint? Give an example.
4. What is the difference between ON DELETE CASCADE and ON DELETE SET NULL?
5. Why is AUTO_INCREMENT used in a table?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—10: Joins

Aim:

Consider following table
students (id, name, age, department, email), courses (course_id, course_name, duration, fees), enrollments (enroll_id, student_id, course_id, enroll_date)
Perform:

1. Inner join
2. Full outer join
3. Left join
4. Right join

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the concept of joins and their importance in relational databases.
2. Differentiate between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.
3. Retrieve data from multiple related tables using join operations.
4. Establish relationships between tables using common columns.
5. Analyze combined data from students, courses, and enrollments tables.

Theory:

1. Introduction to Joins

In relational databases, data is often stored in multiple tables to avoid redundancy and maintain normalization. To retrieve combined data from these related tables, SQL provides **Join operations**.

A **Join** is used to combine rows from two or more tables based on a related column between them.

In this experiment, we perform joins on:

- `students (id, name, age, department, email)`
- `courses (course_id, course_name, duration, fees)`
- `enrollments (enroll_id, student_id, course_id, enroll_date)`

Relationship Structure:

- `students.id` → referenced by `enrollments.student_id`
- `courses.course_id` → referenced by `enrollments.course_id`

2. Why Joins Are Required

Without joins:

- Data remains separated in different tables.
- It is not possible to retrieve complete information (e.g., student name with course name).

With joins:

- Related data from multiple tables can be displayed together.
- Complex queries and reports can be generated.

Example Requirement:
Display student name along with course name and enrollment date.

This requires combining three tables.

3. Types of Joins

3.1 INNER JOIN

An **INNER JOIN** returns only those records that have matching values in both tables.

Syntax:

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

Working Concept:

```
Table A        Table B
   1              1
   2              2
   3              4

Result (INNER JOIN)
   1
   2
```

Only matching records are displayed.

In this experiment:

- Only students who have enrollments will appear.

3.2 LEFT JOIN (LEFT OUTER JOIN)

A **LEFT JOIN** returns:

- All records from the left table
- Matching records from the right table
- NULL values where there is no match

Syntax:

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.column = table2.column;
```

Concept:

```
Left Table      Right Table
   1               1
   2               2
   3

Result:
   1
   2
   3 (NULL)
```

In this experiment:

- All students will appear.
- Students without enrollments will show NULL for course details.

3.3 RIGHT JOIN (RIGHT OUTER JOIN)

A **RIGHT JOIN** returns:

- All records from the right table
- Matching records from the left table
- NULL where no match exists

Syntax:

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

Concept:

```
Left Table      Right Table
   1               1
   2               2
                   4

Result:
   1
   2
   NULL (4)
```

In this experiment:

- All courses will appear.
- Courses without students enrolled will show NULL for student details.

3.4 FULL OUTER JOIN

A **FULL OUTER JOIN** returns:

- All records from both tables
- Matching records combined
- Non-matching records filled with NULL

Standard SQL Syntax:

```
SELECT columns
FROM table1
FULL OUTER JOIN table2
ON table1.column = table2.column;
```

Concept:

```
Left Table     Right Table
   1              1
   2
                  3

Result:
   1
   2
   3
```

Important Note:

MySQL does not directly support FULL OUTER JOIN.
It can be achieved using:

```
LEFT JOIN
UNION
RIGHT JOIN
```

4. Join Between Three Tables

To combine students, enrollments, and courses:

```
students
    |
    | (id = student_id)
    v
enrollments
    |
    | (course_id)
    v
courses
```

This allows retrieval of:

- Student Name
- Course Name
- Enrollment Date

5. Comparison of Join Types

Join Type	Returns Matching Records	Returns Non-Matching Left	Returns Non-Matching Right
INNER JOIN	Yes	No	No
LEFT JOIN	Yes	Yes	No
RIGHT JOIN	Yes	No	Yes
FULL OUTER JOIN	Yes	Yes	Yes

6. Importance of Joins

Joins are important because they:

- Combine related data from multiple tables
- Maintain normalized database structure
- Enable complex queries and reports
- Reduce redundancy
- Support real-world database applications

Code & Output:

(a)

```
mysql> USE College_DB;
Database changed
mysql>
mysql> -- ====================================
mysql> -- 1. INNER JOIN
mysql> -- ====================================
mysql>
mysql> SELECT s.id,
    ->        s.name,
    ->        c.course_name
    -> FROM students s
    -> INNER JOIN enrollments_new e
    -> ON s.id = e.student_id
    -> INNER JOIN course_details c
    -> ON e.course_id = c.course_id;
+----+----------------+-------------+
| id | name           | course_name |
+----+----------------+-------------+
|  1 | Krish Samanta  | BSc CS      |
|  2 | Riddhi Mestry  | BSc CS      |
|  4 | Vedant Gawade  | MSc CS      |
|  5 | Ganesh Yadav   | MSc IT      |
|  6 | Ritesh Gupta   | BSc CS      |
|  7 | Rishi Gupta    | BCA         |
|  9 | Sanchita Tiwari | MSc CS     |
| 10 | Abhishek Khatri | MSc IT     |
+----+----------------+-------------+
8 rows in set (0.00 sec)
```

(b)

```
mysql> -- ====================================
mysql> -- 2. FULL OUTER JOIN
mysql> -- (MySQL does not support FULL OUTER JOIN directly)
mysql> -- So we use UNION of LEFT and RIGHT JOIN
mysql> -- ====================================
mysql>
mysql> SELECT s.id,
    ->        s.name,
    ->        c.course_name
    -> FROM students s
    -> LEFT JOIN enrollments_new e
    -> ON s.id = e.student_id
    -> LEFT JOIN course_details c
    -> ON e.course_id = c.course_id
    ->
    -> UNION
    ->
    -> SELECT s.id,
    ->        s.name,
    ->        c.course_name
    -> FROM students s
    -> RIGHT JOIN enrollments_new e
    -> ON s.id = e.student_id
    -> RIGHT JOIN course_details c
    -> ON e.course_id = c.course_id;
+------+----------------+-------------+
| id   | name           | course_name |
+------+----------------+-------------+
|    1 | Krish Samanta  | BSc CS      |
|    2 | Riddhi Mestry  | BSc CS      |
|    4 | Vedant Gawade  | MSc CS      |
|    5 | Ganesh Yadav   | MSc IT      |
|    6 | Ritesh Gupta   | BSc CS      |
|    7 | Rishi Gupta    | BCA         |
|    9 | Sanchita Tiwari | MSc CS     |
|   10 | Abhishek Khatri | MSc IT     |
| NULL | NULL           | BSc IT      |
+------+----------------+-------------+
9 rows in set (0.01 sec)
```

(c)

```
mysql> -- ====================================
mysql> -- 3. LEFT JOIN
mysql> -- ====================================
mysql>
mysql> SELECT s.id,
    ->        s.name,
    ->        c.course_name
    -> FROM students s
    -> LEFT JOIN enrollments_new e
    -> ON s.id = e.student_id
    -> LEFT JOIN course_details c
    -> ON e.course_id = c.course_id;
+----+----------------+-------------+
| id | name           | course_name |
+----+----------------+-------------+
|  1 | Krish Samanta  | BSc CS      |
|  2 | Riddhi Mestry  | BSc CS      |
|  4 | Vedant Gawade  | MSc CS      |
|  5 | Ganesh Yadav   | MSc IT      |
|  6 | Ritesh Gupta   | BSc CS      |
|  7 | Rishi Gupta    | BCA         |
|  9 | Sanchita Tiwari | MSc CS     |
| 10 | Abhishek Khatri | MSc IT     |
+----+----------------+-------------+
8 rows in set (0.00 sec)
```

निर्मलस्नेह उत्तम सेवाधर्म

(d)

```
mysql> -- ====================================
mysql> -- 4. RIGHT JOIN
mysql> -- ====================================
mysql>
mysql> SELECT s.id,
    ->        s.name,
    ->        c.course_name
    -> FROM students s
    -> RIGHT JOIN enrollments_new e
    -> ON s.id = e.student_id
    -> RIGHT JOIN course_details c
    -> ON e.course_id = c.course_id;
+------+-----------------+-------------+
| id   | name            | course_name |
+------+-----------------+-------------+
|    1 | Krish Samanta   | BSc CS      |
|    2 | Riddhi Mestry   | BSc CS      |
|    6 | Ritesh Gupta    | BSc CS      |
|    7 | Rishi Gupta     | BCA         |
| NULL | NULL            | BSc IT      |
| NULL | NULL            | BSc IT      |
|    4 | Vedant Gawade   | MSc CS      |
|    9 | Sanchita Tiwari | MSc CS      |
|    5 | Ganesh Yadav    | MSc IT      |
|   10 | Abhishek Khatri | MSc IT      |
+------+-----------------+-------------+
10 rows in set (0.00 sec)
```

Learning Outcomes:

1. Write SQL queries to combine data from multiple tables using joins.
2. Apply INNER, LEFT, RIGHT, and FULL OUTER joins appropriately.
3. Interpret query results involving matched and unmatched records.
4. Retrieve meaningful information from relational database systems.
5. Design efficient multi-table queries for real-world database applications.

Course Outcomes:

This experiment enhances students' understanding of relational database concepts by enabling them to combine data from multiple related tables using different types of joins. It strengthens their ability to retrieve meaningful information through INNER, LEFT, RIGHT, and FULL OUTER JOIN operations and to interpret matched and unmatched records correctly. The practical knowledge gained supports the overall course objective of designing efficient multi-table queries and implementing normalized database systems for real-world applications.

Conclusion:

This experiment helped in understanding how to combine data from multiple tables using different types of joins such as INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN. Joins are essential for retrieving related data stored across multiple tables.

Viva Questions:

1. What is the difference between INNER JOIN and LEFT JOIN?
2. Why does MySQL not directly support FULL OUTER JOIN? How can it be implemented?
3. What happens if there is no matching record in a JOIN condition?
4. How are tables related in a JOIN operation?
5. What is the difference between LEFT JOIN and RIGHT JOIN?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—11: Subqueries

Aim:

Perform the following subqueries:

1. Find the names of students who are enrolled in a course with fees greater than 50,000.

2. Find student names who enrolled in the course with the highest fee

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the concept of subqueries in SQL.
2. Differentiate between single-row and multi-row subqueries.
3. Use subqueries within the WHERE clause for conditional filtering.
4. Combine aggregate functions with subqueries.
5. Retrieve data from multiple related tables using nested queries.

Theory:

1. Introduction to Subqueries

A **subquery** is a query written inside another SQL query. It is also called a **nested query**.

A subquery:

- Executes first.
- Returns a result.
- The outer query uses that result to produce the final output.

Subqueries are generally used inside:

- `WHERE` clause
- `FROM` clause
- `SELECT` clause

In this experiment, subqueries are used to:

- Filter students based on course fees.
- Find students enrolled in the course with the highest fee.

2. Basic Structure of a Subquery

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name operator
      (SELECT column_name FROM table_name WHERE condition);
```

Important Points:

1. The subquery is enclosed in parentheses.
2. The inner query executes first.
3. The outer query uses the result returned by the inner query.

3. Types of Subqueries

3.1 Single-Row Subquery

- Returns only one value.
- Used with operators like =, >, <.

Example Concept:

Find course with maximum fee:

```
SELECT MAX(fees) FROM courses;
```

This returns a single value.

3.2 Multi-Row Subquery

- Returns multiple values.
- Used with operators like `IN`, `ANY`, `ALL`.

Example Concept:

```
SELECT course_id
FROM courses
WHERE fees > 50000;
```

This may return multiple course IDs.

4. Subquery for Fees Greater Than 50,000

Step-by-step logic:

1. Find course IDs where fee > 50,000.
2. Find students enrolled in those course IDs.

Conceptual Representation:

```
Step 1: courses
course_id | fees
-----------------
1         | 60000
2         | 45000
3         | 70000

Subquery result → course_id 1 and 3

Step 2: enrollments
student_id | course_id
-----------------------
5          | 1
8          | 3
```

Final result:
Students with ID 5 and 8.

5. Subquery Using MAX() Function

To find students enrolled in the course with the highest fee:

1. Find maximum fee using `MAX(fees)`.
2. Find course having that fee.
3. Find students enrolled in that course.

Logical Flow:

```
Find MAX(fees)
      ↓
Find course_id with that fee
      ↓
Find students enrolled in that course
```

6. Subquery Execution Order

Execution happens in the following order:

```
Inner Query → Outer Query → Final Result
```

Example:

```
SELECT name
FROM students
WHERE id IN
      (SELECT student_id
       FROM enrollments
       WHERE course_id = 3);
```

The subquery runs first and returns student IDs.
The outer query then retrieves student names.

7. Importance of Subqueries

Subqueries are important because they:

- Break complex problems into smaller queries.
- Improve readability of complex conditions.
- Allow use of aggregate functions inside conditions.
- Help retrieve filtered data based on calculated results.
- Support advanced database querying in real-world applications.

Code & Output:

(a)

```
mysql> -- ====================================
mysql> -- 1. Students enrolled in courses with fees > 50,000
mysql> -- ====================================
mysql>
mysql> SELECT name
    -> FROM students
    -> WHERE id IN (
    ->     SELECT student_id
    ->     FROM enrollments_new
    ->     WHERE course_id IN (
    ->         SELECT course_id
    ->         FROM course_details
    ->         WHERE course_fees > 50000
    ->     )
    -> );
+-----------------+
| name            |
+-----------------+
| Krish Samanta   |
| Riddhi Mestry   |
| Vedant Gawade   |
| Ganesh Yadav    |
| Ritesh Gupta    |
| Rishi Gupta     |
| Sanchita Tiwari |
| Abhishek Khatri |
+-----------------+
8 rows in set (0.01 sec)
```

(b)

```
mysql> -- ====================================
mysql> -- 2. Students enrolled in course with highest fee
mysql> -- ====================================
mysql>
mysql> SELECT name
    -> FROM students
    -> WHERE id IN (
    ->     SELECT student_id
    ->     FROM enrollments_new
    ->     WHERE course_id = (
    ->         SELECT course_id
    ->         FROM course_details
    ->         WHERE course_fees = (
    ->             SELECT MAX(course_fees)
    ->             FROM course_details
    ->         )
    ->     )
    -> );
+-----------------+
| name            |
+-----------------+
| Vedant Gawade   |
| Sanchita Tiwari |
+-----------------+
2 rows in set (0.00 sec)
```

निर्मलस्नेह उत्तम सेवाधर्म

Learning Outcomes:

1. Write SQL queries using subqueries to solve complex problems.
2. Use subqueries to filter records based on aggregated conditions.
3. Identify courses based on fee criteria and retrieve related student data.
4. Apply nested queries to retrieve results involving maximum values.
5. Construct efficient and logically structured SQL queries for real-world database scenarios.

Course Outcomes:

This experiment enhances students' ability to write and execute nested SQL queries using subqueries for solving complex database problems. It strengthens their understanding of combining aggregate functions with conditional filtering and retrieving data based on calculated results. The practical knowledge gained supports the overall course objective of

developing advanced querying skills for efficient data retrieval and analysis in relational database systems.

Conclusion:

This experiment demonstrated the use of nested queries to retrieve data based on results returned by another query. Subqueries help solve complex database problems by allowing step-by-step data filtering.

Viva Questions:

1. What is a subquery in SQL?
2. What is the difference between a single-row subquery and a multi-row subquery?
3. Where can subqueries be used in an SQL statement?
4. Why is the IN operator often used with subqueries?
5. Hw does the execution order work in a nested subquery?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—12: String Functions:

Aim:

Perform The Following SQL String Functions
CONCAT(), TRIM(), REPLACE(), REPEAT(), REVERSE(), LCASE()/LOWER(), UCASE()/UPPER(), STRCMP()

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the purpose and usage of string functions in SQL.
2. Apply string manipulation functions such as CONCAT(), TRIM(), and REPLACE().
3. Use case conversion functions like LOWER() and UPPER().
4. Perform string comparison using STRCMP().
5. Manipulate and transform text data stored in database tables effectively.

Theory:

1. Introduction to String Functions in SQL

String functions in SQL are used to manipulate and process textual data stored in database tables. These functions help in:

- Combining strings
- Removing unwanted spaces
- Replacing characters
- Changing letter case
- Comparing strings
- Reversing or repeating text

In this experiment, the following string functions are used:

- `CONCAT()`
- `TRIM()`
- `REPLACE()`
- `REPEAT()`
- `REVERSE()`
- `LCASE()` / `LOWER()`
- `UCASE()` / `UPPER()`
- `STRCMP()`

These functions operate on `VARCHAR`, `CHAR`, or `TEXT` data types.

2. CONCAT()

The `CONCAT()` function combines two or more strings into one string.

Syntax:

```
CONCAT(string1, string2, ...);
```

Example Concept:

```
CONCAT('Hello', ' ', 'World');
```

Result:

```
Hello World
```

It is useful for:

- Combining first name and last name
- Creating formatted output

3. TRIM()

The `TRIM()` function removes leading and trailing spaces from a string.

Syntax:

```
TRIM(string);
```

Example:

```
TRIM('   SQL   ');
```

Result:

```
SQL
```

Importance:

- Cleans unwanted spaces from stored data
- Improves data consistency

4. REPLACE()

The `REPLACE()` function replaces occurrences of a substring with another substring.

Syntax:

```
REPLACE(string, old_value, new_value);
```

Example:

```
REPLACE('I like Java', 'Java', 'Python');
```

Result:

```
I like Python
```

Used for:

- Updating incorrect text
- Changing specific words

5. REPEAT()

The `REPEAT()` function repeats a string a specified number of times.

Syntax:

```
REPEAT(string, number);
```

Example:

```
REPEAT('SQL ', 3);
```

Result:

```
SQL SQL SQL
```

6. REVERSE()

The `REVERSE()` function reverses the characters in a string.

Syntax:

```
REVERSE(string);
```

Example:

```
REVERSE('SQL');
```

Result:

```
LQS
```

7. LOWER() / LCASE()

Both functions convert text into lowercase.

Syntax:

```
LOWER(string);
```

or

```
LCASE(string);
```

Example:

```
LOWER('DATABASE');
```

Result:

```
database
```

8. UPPER() / UCASE()

Both functions convert text into uppercase.

Syntax:

```
UPPER(string);
```

or

```
UCASE(string);
```

Example:

```
UPPER('database');
```

Result:

```
DATABASE
```

9. STRCMP()

The `STRCMP()` function compares two strings.

Syntax:

```
STRCMP(string1, string2);
```

Return Values:

Result	Meaning
0	Strings are equal
1	First string is greater
-1	First string is smaller

Example:

```
STRCMP('Apple', 'Banana');
```

Result:

```
-1
```

Because "Apple" comes before "Banana" alphabetically.

10. Importance of String Functions

String functions are important because they:

- Improve formatting of text data
- Clean inconsistent data
- Help in data transformation
- Support case-sensitive comparisons
- Enhance report generation

These functions are widely used in real-world database applications where textual data manipulation is required.

Code & Output:

```
mysql> -- CONCAT()
mysql> -- ====================================
mysql>
mysql> SELECT CONCAT(name, ' - ', department) AS Student_Info
    -> FROM students;
+----------------------------------------+
| Student_Info                           |
+----------------------------------------+
| Krish Samanta - Computer Science       |
| Riddhi Mestry - Computer Science       |
| Vedant Gawade - Computer Science       |
| Ganesh Yadav - Information Technology  |
| Ritesh Gupta - Computer Science        |
| Rishi Gupta - Computer Applications    |
| Sanchita Tiwari - Computer Science     |
| Abhishek Khatri - Information Technology |
+----------------------------------------+
8 rows in set (0.01 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- TRIM()
mysql> -- ====================================
mysql>
mysql> SELECT TRIM('   Hello World   ') AS Trimmed_Text;
+--------------+
| Trimmed_Text |
+--------------+
| Hello World  |
+--------------+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- REPLACE()
mysql> -- ====================================
mysql>
mysql> SELECT REPLACE(name, 'a', '@') AS Modified_Name
    -> FROM students;
+-----------------+
| Modified_Name   |
+-----------------+
| Krish S@m@nt@   |
| Riddhi Mestry   |
| Ved@nt G@w@de   |
| G@nesh Y@d@v    |
| Ritesh Gupt@    |
| Rishi Gupt@     |
| S@nchit@ Tiw@ri |
| Abhishek Kh@tri |
+-----------------+
8 rows in set (0.00 sec)
```

```
mysql> -- ====================================
mysql> -- REPEAT()
mysql> -- ====================================
mysql>
mysql> SELECT REPEAT(name, 2) AS Repeated_Name
    -> FROM students;
+-------------------------------+
| Repeated_Name                 |
+-------------------------------+
| Krish SamantaKrish Samanta    |
| Riddhi MestryRiddhi Mestry    |
| Vedant GawadeVedant Gawade    |
| Ganesh YadavGanesh Yadav      |
| Ritesh GuptaRitesh Gupta      |
| Rishi GuptaRishi Gupta        |
| Sanchita TiwariSanchita Tiwari |
| Abhishek KhatriAbhishek Khatri |
+-------------------------------+
8 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- REVERSE()
mysql> -- ====================================
mysql>
mysql> SELECT REVERSE(name) AS Reversed_Name
    -> FROM students;
+-----------------+
| Reversed_Name   |
+-----------------+
| atnamaS hsirK   |
| yrtseM ihddiR   |
| edawaG tnadeV   |
| vadaY hsenaG    |
| atpuG hsetiR    |
| atpuG ihsiR     |
| irawiT atihcnaS |
| irtahK kehsihbA |
+-----------------+
8 rows in set (0.00 sec)
```

```
mysql> -- ====================================
mysql> -- LCASE() / LOWER()
mysql> -- ====================================
mysql>
mysql> SELECT LOWER(name) AS Lowercase_Name
    -> FROM students;
+-----------------+
| Lowercase_Name  |
+-----------------+
| krish samanta   |
| riddhi mestry   |
| vedant gawade   |
| ganesh yadav    |
| ritesh gupta    |
| rishi gupta     |
| sanchita tiwari |
| abhishek khatri |
+-----------------+
8 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- UCASE() / UPPER()
mysql> -- ====================================
mysql>
mysql> SELECT UPPER(name) AS Uppercase_Name
    -> FROM students;
+-----------------+
| Uppercase_Name  |
+-----------------+
| KRISH SAMANTA   |
| RIDDHI MESTRY   |
| VEDANT GAWADE   |
| GANESH YADAV    |
| RITESH GUPTA    |
| RISHI GUPTA     |
| SANCHITA TIWARI |
| ABHISHEK KHATRI |
+-----------------+
8 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- STRCMP()
mysql> -- ====================================
mysql>
mysql> SELECT STRCMP('Krish', 'krish') AS Comparison_Result;
+------------------+
| Comparison_Result |
+------------------+
|                 0 |
+------------------+
1 row in set (0.01 sec)
```

Learning Outcomes:

1. Write SQL queries to manipulate string data using built-in functions.
2. Format and clean textual data stored in database tables.
3. Modify and compare string values efficiently.
4. Apply multiple string functions within a single query when required.
5. Use string operations to improve data presentation and analysis in relational databases.

Course Outcomes:

This experiment enhances students' understanding of string manipulation techniques in SQL by enabling them to apply built-in string functions for formatting, cleaning, transforming, and comparing textual data. It strengthens their ability to handle real-world text-based data efficiently within relational databases. The practical skills developed contribute to the overall course objective of writing effective SQL queries for data processing, transformation, and presentation in database applications.

Conclusion:

This experiment introduced various SQL string functions such as CONCAT(), TRIM(), REPLACE(), REPEAT(), REVERSE(), LOWER(), UPPER(), and STRCMP(). These functions help manipulate and format text data stored in database tables.

Viva Questions:

1. What is the purpose of the CONCAT() function in SQL?
2. What is the difference between TRIM() and REPLACE()?
3. What is the output of STRCMP('ABC','ABC') and why?
4. What is the difference between LOWER() and UPPER() functions?
5. How does the REVERSE() function work in SQL?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—13: Math Functions

Aim:

Perform The Following SQL Mathematical Functions
ABS(), CEILING(), FLOOR(), ROUND(), TRUNCATE(), POW(), SQRT(), and RAND().

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the purpose and usage of mathematical functions in SQL.
2. Apply numeric functions such as ABS(), CEILING(), and FLOOR() for data processing.
3. Perform rounding and truncation operations using ROUND() and TRUNCATE().
4. Use power and root functions like POW() and SQRT() in calculations.
5. Generate random numbers using the RAND() function.

Theory:

1. Introduction to Mathematical Functions in SQL

Mathematical functions in SQL are used to perform numeric calculations directly within queries. These functions help in:

- Converting negative values to positive
- Rounding numbers
- Removing decimal places
- Performing exponentiation
- Finding square roots
- Generating random numbers

These functions are commonly used in financial calculations, statistical analysis, grading systems, billing systems, and reporting.

In this experiment, the following mathematical functions are used:

- `ABS()`
- `CEILING()`
- `FLOOR()`
- `ROUND()`
- `TRUNCATE()`
- `POW()`
- `SQRT()`
- `RAND()`

2. ABS() Function

The `ABS()` function returns the absolute value of a number.

Syntax:

```
ABS(number);
```

Example:

```
ABS(-15);
```

Result:

```
15
```

Purpose:

- Removes negative sign.
- Useful in difference calculations.

3. CEILING() Function

The `CEILING()` function returns the smallest integer greater than or equal to a number.

Syntax:

```
CEILING(number);
```

Example:

```
CEILING(4.2);
```

Result:

```
5
```

Explanation:

- Always rounds upward.

4. FLOOR() Function

The `FLOOR()` function returns the largest integer less than or equal to a number.

Syntax:

```
FLOOR(number);
```

Example:

```
FLOOR(4.9);
```

Result:

```
4
```

Explanation:

- Always rounds downward.

5. ROUND() Function

The `ROUND()` function rounds a number to a specified number of decimal places.

Syntax:

```
ROUND(number, decimal_places);
```

Example:

```
ROUND(4.567, 2);
```

Result:

```
4.57
```

Explanation:

- Rounds based on standard rounding rules.

6. TRUNCATE() Function

The `TRUNCATE()` function removes decimal places without rounding.

Syntax:

```
TRUNCATE(number, decimal_places);
```

Example:

```
TRUNCATE(4.567, 2);
```

Result:

```
4.56
```

Difference from ROUND():

- `ROUND()` may increase value.
- `TRUNCATE()` simply cuts off decimals.

7. POW() Function

The `POW()` function returns a number raised to a specified power.

Syntax:

```
POW(base, exponent);
```

Example:

```
POW(2, 3);
```

Result:

```
8
```

8. SQRT() Function

The `SQRT()` function returns the square root of a number.

Syntax:

```
SQRT(number);
```

Example:

```
SQRT(25);
```

Result:

```
5
```

9. RAND() Function

The `RAND()` function generates a random number between 0 and 1.

Syntax:

```
RAND();
```

Example Output:

```
0.734829
```

Features:

- Generates a decimal value.
- Used for random selection and simulations.

10. Comparison Between ROUND() and TRUNCATE()

Function	Behavior	Example	Result
ROUND()	Rounds value	ROUND(4.567,2)	4.57
TRUNCATE()	Cuts decimals	TRUNCATE(4.567,2)	4.56

11. Importance of Mathematical Functions

Mathematical functions are important because they:

- Enable calculations within SQL queries
- Support financial and statistical analysis
- Improve reporting accuracy
- Reduce need for external calculations
- Enhance database-based computation

Code & Output:

```
mysql> -- ====================================
mysql> -- ABS()
mysql> -- ====================================
mysql>
mysql> SELECT ABS(-25) AS Absolute_Value;
+----------------+
| Absolute_Value |
+----------------+
|             25 |
+----------------+
1 row in set (0.01 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- CEILING()
mysql> -- ====================================
mysql>
mysql> SELECT CEILING(4.3) AS Ceiling_Value;
+---------------+
| Ceiling_Value |
+---------------+
|             5 |
+---------------+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- FLOOR()
mysql> -- ====================================
mysql>
mysql> SELECT FLOOR(4.9) AS Floor_Value;
+-------------+
| Floor_Value |
+-------------+
|           4 |
+-------------+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- ROUND()
mysql> -- ====================================
mysql>
mysql> SELECT ROUND(5.678, 2) AS Rounded_Value;
+---------------+
| Rounded_Value |
+---------------+
|          5.68 |
+---------------+
1 row in set (0.00 sec)
```

```
mysql> -- ====================================
mysql> -- TRUNCATE()
mysql> -- ====================================
mysql>
mysql> SELECT TRUNCATE(5.678, 2) AS Truncated_Value;
+-----------------+
| Truncated_Value |
+-----------------+
|            5.67 |
+-----------------+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- POW()
mysql> -- ====================================
mysql>
mysql> SELECT POW(2, 3) AS Power_Value;
+-------------+
| Power_Value |
+-------------+
|           8 |
+-------------+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- SQRT()
mysql> -- ====================================
mysql>
mysql> SELECT SQRT(16) AS Square_Root;
+-------------+
| Square_Root |
+-------------+
|           4 |
+-------------+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- RAND()
mysql> -- ====================================
mysql>
mysql> SELECT RAND() AS Random_Number;
+--------------------+
| Random_Number      |
+--------------------+
| 0.6071405053205234 |
+--------------------+
1 row in set (0.01 sec)
```

Learning Outcomes:

1. Execute SQL queries using built-in mathematical functions accurately.
2. Perform numerical transformations and calculations within database queries.
3. Differentiate between rounding and truncation operations.
4. Apply mathematical operations in real-world database scenarios.
5. Integrate numeric functions into SQL queries for data analysis and computation.

Course Outcomes:

This experiment enhances students' ability to perform numerical computations directly within SQL queries using built-in mathematical functions. It strengthens their understanding of rounding, truncation, absolute value handling, exponentiation, and random number generation in relational databases. The practical knowledge gained supports the overall course objective of applying SQL functions for data analysis, computation, and real-world problem-solving within database systems.

Conclusion:

This experiment demonstrated the use of mathematical functions such as ABS(), CEILING(), FLOOR(), ROUND(), TRUNCATE(), POW(), SQRT(), and RAND(). These functions help perform numeric calculations directly within SQL queries.

Viva Questions:

1. What is the purpose of the ABS() function in SQL?
2. What is the difference between CEILING() and FLOOR() functions?
3. How is ROUND() different from TRUNCATE()?
4. What is the use of the POW() and SQRT() functions?
5. What type of value does the RAND() function generate?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—14: Date Functions

Aim:

Perform The Following SQL Date Functions
NOW(),CURDATE(),CURTIME(), EXTRACT(),DATE(), ADDDATE(), DATE_FORMAT()

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the purpose and usage of date and time functions in SQL.
2. Retrieve current date and time using functions like NOW(), CURDATE(), and CURTIME().
3. Extract specific parts (year, month, day, etc.) from date values using EXTRACT().
4. Perform date arithmetic using ADDDATE().
5. Format date values into different display formats using DATE_FORMAT().

Theory:

1. Introduction to Date Functions in SQL

Date functions in SQL are used to handle and manipulate **date and time values** stored in database tables.

These functions help in:

- Retrieving current date and time
- Extracting specific parts of a date
- Performing date calculations
- Formatting date values

In this experiment, the following date functions are used:

- `NOW()`
- `CURDATE()`
- `CURTIME()`
- `EXTRACT()`
- `DATE()`
- `ADDDATE()`
- `DATE_FORMAT()`

These functions are commonly used in attendance systems, billing systems, order tracking, and scheduling applications.

2. NOW() Function

The `NOW()` function returns the current date and time.

Syntax:

```
NOW();
```

Example Output:

```
2025-02-25 14:35:20
```

Format:

```
YYYY-MM-DD HH:MM:SS
```

Use Case:

- Recording login time
- Storing transaction timestamps

3. CURDATE() Function

The `CURDATE()` function returns the current date only.

Syntax:

```
CURDATE();
```

Example Output:

```
2025-02-25
```

Format:

```
YYYY-MM-DD
```

Use Case:

- Recording today's date
- Comparing date fields

4. CURTIME() Function

The `CURTIME()` function returns the current time only.

Syntax:

```
CURTIME();
```

Example Output:

```
14:35:20
```

Format:

```
HH:MM:SS
```

5. EXTRACT() Function

The `EXTRACT()` function retrieves a specific part of a date.

Syntax:

```
EXTRACT(part FROM date);
```

Common Parts:

- YEAR
- MONTH
- DAY
- HOUR
- MINUTE
- SECOND

Example:

```
EXTRACT(YEAR FROM '2025-02-25');
```

Result:

```
2025
```

Use Case:

- Getting year from birthdate
- Generating monthly reports

6. DATE() Function

The `DATE()` function extracts the date portion from a datetime value.

Syntax:

```
DATE(datetime_value);
```

Example:

```
DATE('2025-02-25 14:35:20');
```

Result:

```
2025-02-25
```

Use Case:

- Removing time from datetime field

7. ADDDATE() Function

The `ADDDATE()` function adds a specified number of days to a date.

Syntax:

```
ADDDATE(date, number_of_days);
```

Example:

```
ADDDATE('2025-02-25', 10);
```

Result:

```
2025-03-07
```

Use Case:

- Calculating expiry dates
- Finding due dates

8. DATE_FORMAT() Function

The `DATE_FORMAT()` function formats a date into a specified pattern.

Syntax:

```
DATE_FORMAT(date, format_string);
```

Common Format Specifiers:

Specifier	Meaning
%Y	Year (4 digits)
%m	Month (2 digits)
%d	Day
%M	Month name
%W	Day name

Example:

```
DATE_FORMAT('2025-02-25', '%W, %M %d, %Y');
```

Result:

```
Tuesday, February 25, 2025
```

Use Case:

- Displaying date in readable format
- Generating formatted reports

9. Difference Between NOW(), CURDATE(), and CURTIME()

Function	Returns
NOW()	Current date and time
CURDATE()	Current date only
CURTIME()	Current time only

10. Importance of Date Functions

Date functions are important because they:

- Enable time-based calculations
- Support scheduling systems
- Help in report generation
- Allow extraction of date components
- Improve data formatting and presentation

Code & Output:

```
mysql> -- ====================================
mysql> -- NOW()
mysql> -- ====================================
mysql>
mysql> SELECT NOW() AS Current_Date_Time;
+---------------------+
| Current_Date_Time   |
+---------------------+
| 2026-02-28 04:01:23 |
+---------------------+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- CURDATE()
mysql> -- ====================================
mysql>
mysql> SELECT CURDATE() AS Today_Date;
+------------+
| Today_Date |
+------------+
| 2026-02-28 |
+------------+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- CURTIME()
mysql> -- ====================================
mysql>
mysql> SELECT CURTIME() AS Current_Time_Value;
+--------------------+
| Current_Time_Value |
+--------------------+
| 04:01:23           |
+--------------------+
1 row in set (0.01 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- EXTRACT()
mysql> -- ====================================
mysql>
mysql> SELECT EXTRACT(YEAR FROM NOW()) AS Current_Year;
+--------------+
| Current_Year |
+--------------+
|         2026 |
+--------------+
1 row in set (0.00 sec)
```

```
mysql> -- ====================================
mysql> -- DATE()
mysql> -- ====================================
mysql>
mysql> SELECT DATE(NOW()) AS Only_Date;
+------------+
| Only_Date  |
+------------+
| 2026-02-28 |
+------------+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- ADDDATE()
mysql> -- ====================================
mysql>
mysql> SELECT ADDDATE(CURDATE(), INTERVAL 7 DAY) AS Date_After_7_Days;
+-------------------+
| Date_After_7_Days |
+-------------------+
| 2026-03-07        |
+-------------------+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- DATE_FORMAT()
mysql> -- ====================================
mysql>
mysql> SELECT DATE_FORMAT(NOW(), '%d-%m-%Y') AS Formatted_Date;
+----------------+
| Formatted_Date |
+----------------+
| 28-02-2026     |
+----------------+
1 row in set (0.00 sec)
```

Learning Outcomes:

1. Execute SQL queries using built-in date and time functions accurately.
2. Manipulate and calculate date values within database queries.
3. Extract meaningful components from date fields for reporting purposes.
4. Format date and time values for user-friendly presentation.
5. Apply date functions in real-world database applications such as billing systems, attendance tracking, and scheduling systems.

Course Outcomes:

This experiment enhances students' ability to work with date and time data in relational databases using built-in SQL date functions. It strengthens their understanding of retrieving current date and time, performing date arithmetic, extracting specific components, and formatting dates for meaningful presentation. The practical skills developed support the overall course objective of implementing time-based logic and data manipulation in real-world database applications such as billing systems, attendance tracking, and scheduling systems.

Conclusion:

This experiment helped in understanding how to work with date and time values using functions such as NOW(), CURDATE(), CURTIME(), EXTRACT(), ADDDATE(), and DATE_FORMAT(). These functions are useful for performing time-based calculations and formatting date values.

Viva Questions:

1. What is the difference between NOW(), CURDATE(), and CURTIME()?
2. How does the EXTRACT() function work? Give an example.
3. What is the purpose of the DATE() function?
4. How can you add days to a date in SQL?
5. Why is the DATE_FORMAT() function used? Give an example format specifier.

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—15: Views

Aim:

Perform Queries to:

1. Create a view
2. Update a view
3. Insert values in view
4. Drop view

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the concept and purpose of views in SQL.
2. Create a view based on a SELECT query.
3. Perform data manipulation operations (INSERT, UPDATE) through a view.
4. Identify conditions under which a view is updatable.
5. Remove a view using the DROP VIEW statement.

Theory:

1. Introduction to Views

A **View** in SQL is a virtual table based on the result of a `SELECT` query.

Unlike a physical table:

- A view does not store data separately.
- It displays data stored in one or more base tables.
- It acts as a saved SQL query.

Views are mainly used for:

- Simplifying complex queries
- Providing data security
- Restricting access to specific columns
- Presenting data in a customized format

2. Creating a View

A view is created using the `CREATE VIEW` statement.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example Concept:

If we have a `students` table and want to display only name and email:

```
CREATE VIEW student_view AS
SELECT name, email
FROM students;
```

Now, instead of writing the full SELECT query, we can use:

```
SELECT * FROM student_view;
```

The view behaves like a table.

3. Updating a View

If a view is **updatable**, changes made through the view affect the base table.

Example:

```
UPDATE student_view
SET name = 'Rahul Sharma'
WHERE name = 'Rahul';
```

Important Conditions for Updatable View:

- View must be based on a single table.
- It should not contain:
  - GROUP BY
  - HAVING
  - Aggregate functions
  - DISTINCT
  - JOIN

If these are used, the view becomes non-updatable.

4. Inserting Data Through a View

We can insert values through a view if:

- The view includes all required NOT NULL columns.
- The view is based on a single table.

Example:

```
INSERT INTO student_view (name, email)
VALUES ('Anu', 'anu@gmail.com');
```

This inserts data into the original `students` table.

5. Dropping a View

A view can be removed using the `DROP VIEW` statement.

Syntax:

```
DROP VIEW view_name;
```

Example:

```
DROP VIEW student_view;
```

Important:

- Dropping a view does not delete data from the base table.
- It only removes the virtual table.

6. How Views Work Internally

Conceptual Representation:

```
Base Table (students)
        ↓
     View (student_view)
        ↓
   User Query
```

When a user queries the view:

- MySQL executes the underlying SELECT query.
- The result is returned dynamically.

7. Advantages of Views

Views provide:

- Data abstraction
- Security (hide sensitive columns)
- Simplified complex queries
- Consistency in reporting
- Logical data independence

8. Difference Between Table and View

Feature	Table	View
Stores Data	Yes	No
Physical Storage	Yes	No
Based On	Independent	Based on table
Used For	Storing data	Viewing/filtering data

9. Importance of Views

Views are important because they:

- Restrict user access to sensitive data
- Reduce query complexity
- Improve maintainability
- Provide logical representation of data
- Support secure database design

Code & Output:

(a)

```
mysql> -- ====================================
mysql> -- 1. Create a View
mysql> -- ====================================
mysql>
mysql> CREATE VIEW student_view AS
    -> SELECT id, name, department
    -> FROM students;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> -- Show view data
mysql> SELECT * FROM student_view;
+----+-----------------+-----------------------+
| id | name            | department            |
+----+-----------------+-----------------------+
|  1 | Krish Samanta   | Computer Science      |
|  2 | Riddhi Mestry   | Computer Science      |
|  4 | Vedant Gawade   | Computer Science      |
|  5 | Ganesh Yadav    | Information Technology |
|  6 | Ritesh Gupta    | Computer Science      |
|  7 | Rishi Gupta     | Computer Applications |
|  9 | Sanchita Tiwari | Computer Science      |
| 10 | Abhishek Khatri | Information Technology |
+----+-----------------+-----------------------+
8 rows in set (0.00 sec)
```

(b)

```
mysql> -- ====================================
mysql> -- 2. Update a View
mysql> -- ====================================
mysql>
mysql> UPDATE student_view
    -> SET department = 'Computer Science'
    -> WHERE id = 7;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> -- Verify update
mysql> SELECT * FROM student_view;
+----+----------------+-----------------------+
| id | name           | department            |
+----+----------------+-----------------------+
|  1 | Krish Samanta  | Computer Science      |
|  2 | Riddhi Mestry  | Computer Science      |
|  4 | Vedant Gawade  | Computer Science      |
|  5 | Ganesh Yadav   | Information Technology |
|  6 | Ritesh Gupta   | Computer Science      |
|  7 | Rishi Gupta    | Computer Science      |
|  9 | Sanchita Tiwari | Computer Science      |
| 10 | Abhishek Khatri | Information Technology |
+----+----------------+-----------------------+
8 rows in set (0.00 sec)
```

निर्मलस्नेह उत्तम सेवाधर्म

(c)

```
mysql> -- ====================================
mysql> -- 3. Insert values into View
mysql> -- ====================================
mysql>
mysql> INSERT INTO student_view (id, name, department)
    -> VALUES (3, 'Jay Patil', 'Information Technology');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO student_view (id, name, department)
    -> VALUES (8, 'Anuvesha Singh', 'Computer Applications');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> -- Verify insert
mysql> SELECT * FROM student_view;
+----+-----------------+-----------------------+
| id | name            | department            |
+----+-----------------+-----------------------+
|  1 | Krish Samanta   | Computer Science      |
|  2 | Riddhi Mestry   | Computer Science      |
|  3 | Jay Patil       | Information Technology |
|  4 | Vedant Gawade   | Computer Science      |
|  5 | Ganesh Yadav    | Information Technology |
|  6 | Ritesh Gupta    | Computer Science      |
|  7 | Rishi Gupta     | Computer Science      |
|  8 | Anuvesha Singh  | Computer Applications |
|  9 | Sanchita Tiwari | Computer Science      |
| 10 | Abhishek Khatri | Information Technology |
+----+-----------------+-----------------------+
10 rows in set (0.00 sec)
```

(d)

```
mysql> -- ====================================
mysql> -- 4. Drop View
mysql> -- ====================================
mysql>
mysql> DROP VIEW student_view;
Query OK, 0 rows affected (0.00 sec)
```

Learning Outcomes:

1. Create and manage views for simplified data access.
2. Perform update and insert operations using updatable views.
3. Understand the relationship between base tables and views.
4. Apply views for data abstraction and security.
5. Implement view-based query management in relational database systems.

Course Outcomes:

This experiment enhances students' understanding of views as a mechanism for data abstraction, security, and simplified query management in relational databases. It enables them to create, modify, insert, and drop views while understanding their impact on underlying base tables. The practical skills gained support the overall course objective of designing efficient and secure database systems by implementing logical data representation and controlled data access mechanisms.

Conclusion:

This experiment demonstrated how to create, update, insert into, and drop views. Views help simplify complex queries and provide controlled access to database data.

Viva Questions:

1. What is a view in SQL and how is it different from a table?
2. Under what conditions is a view updatable?
3. What happens to the base table when we update data through a view?
4. Can a view be created using multiple tables? If yes, will it always be updatable?
5. What is the purpose of using views in database systems?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—16: Views and Transactions

Aim:

Implement view-based operations and transactions using COMMIT and ROLLBACK

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the concept of database transactions in SQL.
2. Learn how views can be used to perform controlled data operations.
3. Implement transaction control commands such as COMMIT and ROLLBACK.
4. Understand how transactions ensure data consistency and reliability.
5. Perform view-based data manipulation within a transactional environment.

Theory:

1. Introduction to Views and Transactions

In relational database systems, **views** and **transactions** are important mechanisms used to manage data access and maintain data integrity.

- **Views** provide a virtual representation of data from one or more tables.
- **Transactions** ensure that a group of database operations is executed as a single logical unit.

In this experiment, view-based operations are performed along with transaction control commands such as **COMMIT** and **ROLLBACK**.

2. View-Based Operations

A **view** is a virtual table created from the result of a SQL query. It does not store data itself but displays data from the base tables.

Views help in:

- Simplifying complex queries
- Restricting access to specific columns
- Enhancing database security
- Providing customized data representation

Example Concept:

```
CREATE VIEW student_view AS
SELECT name, department, email
FROM students;
```

Once the view is created, it can be used like a table for retrieving or modifying data.

Example:

```
SELECT * FROM student_view;
```

If the view is **updatable**, changes made through the view affect the underlying base table.

3. Introduction to Transactions

A **transaction** is a sequence of one or more SQL operations executed as a single unit of work.

A transaction ensures that:

- Either all operations succeed
- Or none of them are applied

This property is essential for maintaining database consistency.

4. Transaction Control Commands

SQL provides **Transaction Control Language (TCL)** commands to manage transactions.

The main commands are:

- `START TRANSACTION / BEGIN`
- `COMMIT`
- `ROLLBACK`

5. COMMIT Statement

The `COMMIT` statement permanently saves all changes made during the current transaction.

Syntax:

```
COMMIT;
```

Working Concept:

```
Start Transaction
        ↓
Perform INSERT / UPDATE / DELETE
        ↓
COMMIT
        ↓
Changes permanently saved
```

Once committed, the changes cannot be undone using ROLLBACK.

Example:

```
START TRANSACTION;
UPDATE students SET department = 'IT' WHERE id = 5;
COMMIT;
```

6. ROLLBACK Statement

The `ROLLBACK` statement cancels all changes made during the current transaction and restores the database to the previous state.

Syntax:

```
ROLLBACK;
```

Working Concept:

```
Start Transaction
        ↓
Perform INSERT / UPDATE / DELETE
        ↓
ROLLBACK
        ↓
Changes cancelled
```

Example:

```
START TRANSACTION;
UPDATE students SET department = 'IT' WHERE id = 5;
ROLLBACK;
```

In this case, the update operation will not be saved.

7. View-Based Transactions

When operations are performed through a view, they can also be controlled using transactions.

Example Process:

```
Create View
      ↓
Start Transaction
      ↓
Update/Insert through View
      ↓
COMMIT or ROLLBACK
```

If `COMMIT` is used:

- Changes are saved in the base table.

If `ROLLBACK` is used:

- Changes are cancelled.

8. Example Conceptual Workflow

```
Students Table
       ↓
   Student View
       ↓
START TRANSACTION
       ↓
Update through View
       ↓
COMMIT / ROLLBACK
```

This ensures safe and controlled modification of data.

9. Importance of Transactions

Transactions are important because they:

- Maintain data consistency
- Prevent partial updates
- Handle errors effectively
- Ensure reliability of database operations
- Support safe multi-step operations

Code & Output:

```
mysql> -- ====================================
mysql> -- Create a View
mysql> -- ====================================
mysql>
mysql> CREATE VIEW student_basic_view AS
    -> SELECT id, name, age, department
    -> FROM students;
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> -- Display view
mysql> SELECT * FROM student_basic_view;
+----+-----------------+------+------------------------+
| id | name            | age  | department             |
+----+-----------------+------+------------------------+
|  1 | Krish Samanta   |   20 | Computer Science       |
|  2 | Riddhi Mestry   |   19 | Computer Science       |
|  3 | Jay Patil       | NULL | Information Technology |
|  4 | Vedant Gawade   |   20 | Computer Science       |
|  5 | Ganesh Yadav    |   18 | Information Technology |
|  6 | Ritesh Gupta    |   21 | Computer Science       |
|  7 | Rishi Gupta     |   22 | Computer Science       |
|  8 | Anuvesha Singh  | NULL | Computer Applications  |
|  9 | Sanchita Tiwari |   23 | Computer Science       |
| 10 | Abhishek Khatri |   19 | Information Technology |
+----+-----------------+------+------------------------+
10 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- ====================================
mysql> -- Update data through View
mysql> -- ====================================
mysql>
mysql> UPDATE student_basic_view
    -> SET department = 'Computer Science'
    -> WHERE id = 5;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> -- Verify update
mysql> SELECT * FROM students WHERE id = 5;
+----+--------------+------+------------------+---------+----------+------------+------------------+
| id | name         | age  | email            | address | category | phone      | department       |
+----+--------------+------+------------------+---------+----------+------------+------------------+
|  5 | Ganesh Yadav |   18 | ganesh@gmail.com | Mumbai  | Student  | 9876543214 | Computer Science |
+----+--------------+------+------------------+---------+----------+------------+------------------+
1 row in set (0.00 sec)
```

```
mysql> -- ====================================
mysql> -- Start Transaction
mysql> -- ====================================
mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE students
    -> SET age = 25
    -> WHERE id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> -- Check temporary change
mysql> SELECT * FROM students WHERE id = 2;
+----+---------------+------+------------------+---------+----------+------------+------------------+
| id | name          | age  | email            | address | category | phone      | department       |
+----+---------------+------+------------------+---------+----------+------------+------------------+
|  2 | Riddhi Mestry |   25 | riddhi@gmail.com | Mumbai  | Student  | 9876543211 | Computer Science |
+----+---------------+------+------------------+---------+----------+------------+------------------+
1 row in set (0.00 sec)

mysql>
mysql> -- Undo change
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Verify rollback
mysql> SELECT * FROM students WHERE id = 2;
+----+---------------+------+------------------+---------+----------+------------+------------------+
| id | name          | age  | email            | address | category | phone      | department       |
+----+---------------+------+------------------+---------+----------+------------+------------------+
|  2 | Riddhi Mestry |   19 | riddhi@gmail.com | Mumbai  | Student  | 9876543211 | Computer Science |
+----+---------------+------+------------------+---------+----------+------------+------------------+
1 row in set (0.00 sec)
```

```
mysql> -- ====================================
mysql> -- Transaction with COMMIT
mysql> -- ====================================
mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE students
    -> SET age = 24
    -> WHERE id = 3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> -- Save change permanently
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> -- Verify commit
mysql> SELECT * FROM students WHERE id = 3;
+----+-----------+------+-------+---------+----------+-------+-----------------------+
| id | name      | age  | email | address | category | phone | department            |
+----+-----------+------+-------+---------+----------+-------+-----------------------+
|  3 | Jay Patil |   24 | NULL  | NULL    | Student  | NULL  | Information Technology |
+----+-----------+------+-------+---------+----------+-------+-----------------------+
1 row in set (0.00 sec)
```

Learning Outcomes:

1. Execute SQL transactions to control database operations safely.
2. Use COMMIT to permanently save changes in the database.
3. Use ROLLBACK to undo changes when an error occurs.
4. Perform insert or update operations through views within transactions.
5. Apply transaction control techniques to maintain data integrity in relational databases.

Course Outcomes:

This experiment enhances students' understanding of transaction management and controlled data manipulation in relational databases. It enables them to perform operations through views while applying transaction control commands such as COMMIT and ROLLBACK to ensure data consistency and reliability. The practical knowledge gained supports the overall course objective of implementing secure and reliable database operations using transaction control mechanisms in real-world database applications.

Conclusion:

This experiment provided knowledge about transaction control using COMMIT and ROLLBACK. It showed how database operations can be controlled and how transactions ensure data consistency and reliability.

Viva Questions:

1. What is a transaction in a database system?
2. What is the purpose of the COMMIT statement?
3. What does the ROLLBACK command do in SQL?
4. Can database operations performed through a view be controlled using transactions? Explain.
5. What is the difference between COMMIT and ROLLBACK?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]

Experiment No.—17: Complex Queries using ANY, ALL, EXISTS

Aim:

Use these logical operators in subqueries for conditional evaluations

Tools Used:

Programming Language: SQL (Structured Query Language)

Compiler/IDE: MySQL Workbench (GUI-based IDE for executing SQL queries), MySQL Command Line Client or phpMyAdmin (via XAMPP Control Panel)

Learning Objectives:

1. Understand the concept of complex queries using subquery operators.
2. Learn how to use the ANY, ALL, and EXISTS operators in SQL.
3. Apply logical operators in subqueries for conditional evaluation.
4. Compare values returned by subqueries using different comparison operators.
5. Construct advanced SQL queries for complex data retrieval.

Theory:

1. Introduction to Complex Queries

In SQL, complex queries are used when simple conditions are not sufficient to retrieve the required data. These queries often involve **subqueries** combined with logical operators to evaluate conditions based on results returned from another query.

Subqueries allow SQL to perform step-by-step evaluation where:

- The **inner query executes first**
- The **outer query uses the result of the inner query**

In this experiment, the following operators are used with subqueries:

- `ANY`
- `ALL`
- `EXISTS`

These operators help in performing **conditional evaluations on sets of values returned by subqueries**.

2. Subqueries in SQL

A **subquery** is a query nested inside another SQL query.

Basic Structure:

```
SELECT column_name
FROM table_name
WHERE column_name operator
      (SELECT column_name FROM table_name WHERE condition);
```

Execution Flow:

```
Inner Query → Result Set
       ↓
Outer Query uses result
       ↓
Final Output
```

Subqueries can return:

- A single value
- Multiple values
- Entire rows

3. ANY Operator

The `ANY` operator compares a value with **any value returned by a subquery**.

Condition is **true if at least one value satisfies the comparison**.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name operator ANY
      (subquery);
```

Example Concept:

```
Subquery Result: 20000, 30000, 40000

Condition:
salary > ANY (values)

If salary > 20000 → TRUE
```

Thus, if the condition is true for **any one value**, the record is selected.

4. ALL Operator

The `ALL` operator compares a value with **all values returned by the subquery**.

The condition is **true only if it satisfies the comparison with every value**.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name operator ALL
      (subquery);
```

Example Concept:

```
Subquery Result: 20000, 30000, 40000

Condition:
salary > ALL (values)

If salary > 40000 → TRUE
```

Thus, the value must satisfy the condition **for every value returned by the subquery**.

5. EXISTS Operator

The `EXISTS` operator checks whether the subquery returns **any records**.

If the subquery returns at least one row, the condition becomes **true**.

Syntax:

```
SELECT column_name
FROM table_name
WHERE EXISTS (subquery);
```

Example Concept:

```
Subquery Result:
student_id | course_id
-----------------------
5          | 101

Since at least one row exists → condition TRUE
```

The outer query will execute.

6. Comparison Between ANY, ALL, and EXISTS

Operator	Meaning	Condition Result
ANY	True if comparison matches at least one value	At least one value satisfies condition
ALL	True if comparison matches all values	All values satisfy condition
EXISTS	Checks existence of records	True if subquery returns rows

7. Conceptual Workflow

Example Scenario:

```
Students Table
      ↓
Enrollments Table
      ↓
Courses Table
```

A subquery can be used to:

- Find courses with high fees
- Compare student records with course records
- Check if enrollment records exist

The logical operators then evaluate conditions on the returned results.

8. Importance of ANY, ALL, and EXISTS

These operators are important because they:

- Enable complex conditional comparisons
- Improve query flexibility
- Allow evaluation of multiple values returned by subqueries
- Help retrieve specific data from related tables
- Support advanced database query logic

Code & Output:

(a) ANY Operator

```
mysql> -- ====================================
mysql> -- (a) ANY Operator
mysql> -- Find students enrolled in courses with fees greater than ANY course fee above 7000
mysql> -- ====================================
mysql>
mysql> SELECT name
    -> FROM students
    -> WHERE id = ANY (
    ->     SELECT student_id
    ->     FROM enrollments_new
    ->     WHERE course_id = ANY (
    ->         SELECT course_id
    ->         FROM course_details
    ->         WHERE course_fees > 70000
    ->     )
    -> );
+-----------------+
| name            |
+-----------------+
| Krish Samanta   |
| Riddhi Mestry   |
| Jay Patil       |
| Vedant Gawade   |
| Ganesh Yadav    |
| Ritesh Gupta    |
| Anuvesha Singh  |
| Sanchita Tiwari |
| Abhishek Khatri |
+-----------------+
9 rows in set (0.01 sec)
```

(b) ALL Operator

```
mysql> -- ====================================
mysql> -- (b) ALL Operator
mysql> -- Find courses whose fees are greater than ALL courses with duration = 24
mysql> -- ====================================
mysql>
mysql> SELECT course_name, course_fees
    -> FROM course_details
    -> WHERE course_fees > ALL (
    ->     SELECT course_fees
    ->     FROM course_details
    ->     WHERE duration = 24
    -> );
Empty set (0.00 sec)
```

(c) EXISTS Operator

```
mysql> -- ====================================
mysql> -- (c) EXISTS Operator
mysql> -- Find students who are enrolled in any course
mysql> -- ====================================
mysql>
mysql> SELECT name
    -> FROM students s
    -> WHERE EXISTS (
    ->     SELECT *
    ->     FROM enrollments_new e
    ->     WHERE s.id = e.student_id
    -> );
+-----------------+
| name            |
+-----------------+
| Krish Samanta   |
| Riddhi Mestry   |
| Jay Patil       |
| Vedant Gawade   |
| Ganesh Yadav    |
| Ritesh Gupta    |
| Rishi Gupta     |
| Anuvesha Singh  |
| Sanchita Tiwari |
| Abhishek Khatri |
+-----------------+
10 rows in set (0.00 sec)
```

Learning Outcomes:

1. Write SQL queries using ANY, ALL, and EXISTS with subqueries.
2. Analyze and filter data using advanced conditional logic.
3. Differentiate between the behavior of ANY, ALL, and EXISTS.
4. Retrieve meaningful information from relational databases using complex queries.
5. Apply advanced SQL techniques to solve real-world database problems.

Course Outcomes:

This experiment enhances students' ability to construct advanced SQL queries using subquery operators such as ANY, ALL, and EXISTS for conditional evaluation. It strengthens their understanding of complex query logic and enables them to retrieve data based on comparisons with multiple values returned by subqueries. The practical knowledge gained supports the overall course objective of developing advanced querying skills for efficient data retrieval and analysis in relational database systems.

Conclusion:

This experiment helped in understanding advanced SQL queries using the ANY, ALL, and EXISTS operators. These operators are useful for performing conditional evaluations on results returned by subqueries.

Viva Questions:

1. What is the purpose of using ANY, ALL, and EXISTS operators in SQL?
2. What is the difference between ANY and ALL in SQL queries?
3. How does the EXISTS operator work in a subquery?
4. Can the EXISTS operator return TRUE if the subquery returns only one row? Explain.
5. What is the difference between IN and EXISTS operators in SQL?

For Faculty use:

Correction Parameters	Formative Assessment[40%]	Timely Completion of Practical[40%]	Attendance Learning Attitude[20%]